

Private Network Interconnection (NAT, VPN)

20.1 Introduction

Previous chapters describe an internet as a single-level abstraction that consists of networks interconnected by routers. This chapter considers an alternative — a two-level internet architecture in which each organization has a private internet and a central internet interconnects them.

The chapter examines technologies used with a two-level architecture. One solves the pragmatic problem of limited address space, and the other offers increased functionality in the form of *privacy* that prevents outsiders from viewing the data.

20.2 Private And Hybrid Networks

One of the major drawbacks of a single-level internet architecture is the lack of privacy. If an organization comprises multiple sites, the contents of datagrams that travel across the Internet between the sites can be viewed by outsiders because they pass across networks owned by other organizations. A two-level architecture distinguishes between *internal* and *external* datagrams (i.e., datagrams sent between two computers within an organization and datagrams sent between a computer in the organization and a computer in another organization). The goal is to keep internal datagrams *private*, while still allowing external communication.

The easiest way to guarantee privacy among an organization's computers consists of building a completely isolated *private internet*, which is usually referred to as a

private network. That is, an organization builds its own TCP/IP internet separate from the global Internet. A private network uses routers to interconnect networks at each site, and leased digital circuits to interconnect the sites. All data remains private because no outsiders have access to any part of a private network. Furthermore, because the private network is isolated from the global Internet, it can use arbitrary IP addresses.

Of course, complete isolation is not always desirable. Thus, many organizations choose a *hybrid network* architecture that combines the advantages of private networking with the advantages of global Internet connectivity. That is, the organization uses globally valid IP addresses and connects each site to the Internet. The advantage is that hosts in the organization can access the global Internet when needed, but can be assured of privacy when communicating internally. For example, consider the hybrid architecture illustrated by Figure 20.1 in which an organization has a private network that interconnects two sites and each site has a connection to the Internet.

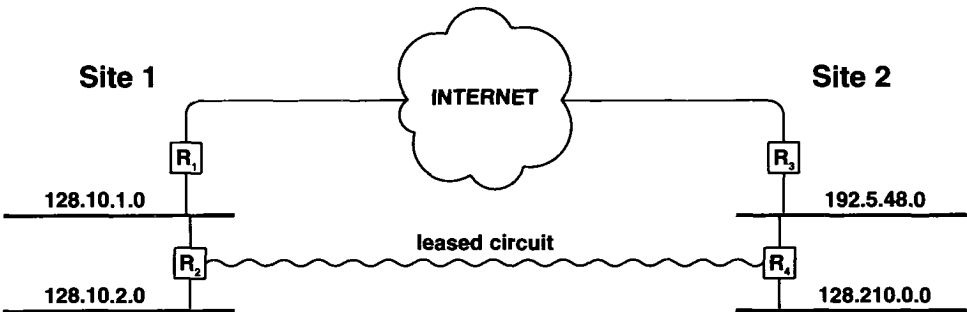


Figure 20.1 An example of a hybrid network. In addition to a leased circuit that interconnects the two sites, each has a connection to the global Internet.

In the figure, a leased circuit between routers R_2 and R_4 provides privacy for inter-site traffic. Thus, routing at each site is arranged to send traffic across the leased circuit rather than across the global Internet.

20.3 A Virtual Private Network (VPN)

The chief disadvantage of either a completely private network or a hybrid scheme arises from the high cost: each leased circuit (e.g., a T1 line) is expensive. Consequently, many organizations seek lower-cost alternatives. One way to reduce costs arises from the use of alternative circuit technologies. For example, a common carrier may charge less for a Frame Relay or ATM PVC than for a T-series circuit that has equivalent capacity. Another way to lower costs involves using fewer circuits. Minimum circuit cost is achieved by eliminating all circuits and passing data across the global Internet.

Using the global Internet as an interconnection among sites appears to eliminate the privacy offered by a completely private network. The question becomes:

How can an organization that uses the global Internet to connect its sites keep its data private?

The answer lies in a technology that allows an organization to configure a *Virtual Private Network (VPN)*[†]. A VPN is *private* in the same way as a private network — the technology guarantees that communication between any pair of computers in the VPN remains concealed from outsiders. A VPN is *virtual* because it does not use leased circuits to interconnect sites. Instead, a VPN uses the global Internet to pass traffic from one site to another.

Two basic techniques make a VPN possible: *tunneling* and *encryption*. We have already encountered tunneling in Chapters 17 and 19. VPNs use the same basic idea — they define a tunnel across the global Internet between a router at one site and a router at another, and use *IP-in-IP* encapsulation to forward datagrams across the tunnel.

Despite using the same basic concept, a VPN tunnel differs dramatically from the tunnels described previously. In particular, to guarantee privacy, a VPN encrypts each outgoing datagram before encapsulating it in another datagram for transmission[‡]. Figure 20.2 illustrates the concept.

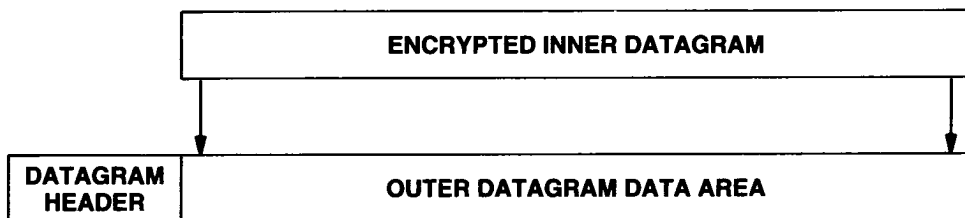


Figure 20.2 Illustration of IP-in-IP encapsulation used with a VPN. To ensure privacy, the inner datagram is encrypted before being sent.

As the figure shows, the entire inner datagram, including the header, is encrypted before being encapsulated. When a datagram arrives over a tunnel, the receiving router decrypts the data area to reproduce the inner datagram, which it then forwards. Although the outer datagram traverses arbitrary networks as it passes across the tunnel, outsiders cannot decode the contents because they do not have the encryption key. Furthermore, even the identity of the original source and destination are hidden because the header of the inner datagram is encrypted as well. Thus, only addresses in the outer datagram header are visible: the source address is the IP address of the router at one end of a tunnel, and the destination address is the IP address of the router at the other end of the tunnel.

[†]The name is a slight misnomer because the technology actually provides a virtual private internet.

[‡]Chapter 32 considers IP security, and discusses the encapsulation used with IPsec.

To summarize:

A Virtual Private Network sends data across the Internet, but encrypts intersite transmissions to guarantee privacy.

20.4 VPN Addressing And Routing

The easiest way to understand VPN addressing and routing is to think of each VPN tunnel as a replacement for a leased circuit in a private network. As in the private network case, a router contains explicit routes for destinations within the organization. However, instead of routing data across a leased lined, a VPN routes the data through a tunnel. For example Figure 20.3 shows the VPN equivalent of the private network architecture from Figure 20.1 along with a routing table for a router that handles tunneling.

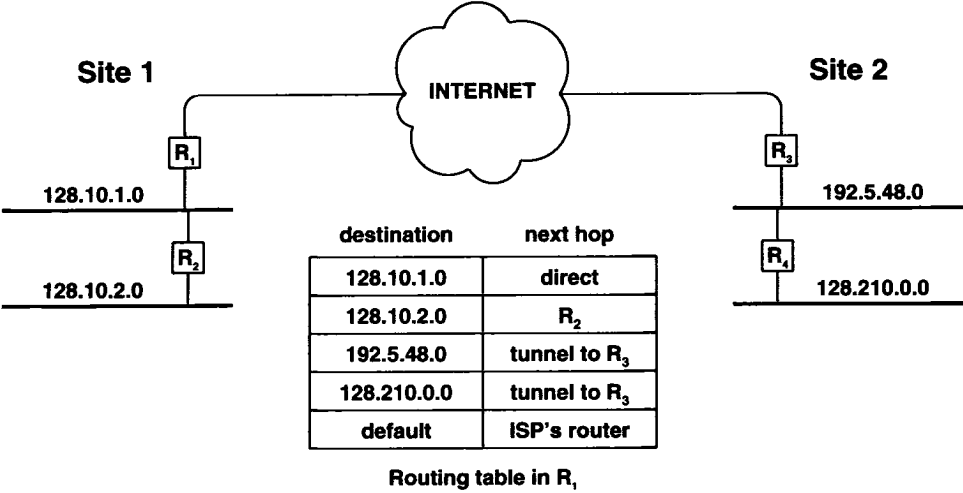


Figure 20.3 A VPN that spans two sites and R₁'s routing table. The tunnel from R₁ to R₃ is configured like a point-to-point leased circuit.

As an example of forwarding in a VPN, consider a datagram sent from a computer on network 128.10.2.0 to a computer on network 128.210.0.0. The sending host forwards the datagram to R₂, which forwards it to R₁. According to the routing table in R₁, the datagram must be sent across the tunnel to R₃. Therefore, R₁ encrypts the datagram, encapsulates it in the data area of an outer datagram with destination R₃. R₁ then forward the outer datagram through the local ISP and across the Internet. The datagram arrives at R₃, which recognizes it as tunneled from R₁. R₃ decrypts the data area to pro-

duce the original datagram, looks up the destination in its routing table, and forwards the datagram to R_4 for delivery.

20.5 A VPN With Private Addresses

A VPN offers an organization the same addressing options as a private network. If hosts in the VPN do not need general Internet connectivity, the VPN can be configured to use arbitrary IP addresses; if hosts need Internet access, a hybrid addressing scheme can be used. A minor difference is that when private addressing is used, one globally valid IP address is needed at each site for tunneling. Figure 20.4 illustrates the concept.



Figure 20.4 Illustration of addressing for a VPN that interconnects two completely private sites over the global Internet. Computers at each site use private addresses.

As the figure shows, site 1 uses subnet 10.1.0.0/16, while site 2 uses subnet 10.2.0.0/16. Only two globally valid addresses are needed. One is assigned to the connection from router R_1 to the Internet, and the other is assigned to the connection from R_2 to the Internet. Routing tables at the sites specify routes for private addresses; only the VPN tunneling software needs to know about or use the globally valid IP addresses.

VPNs use the same addressing structure as a private network. Hosts in a completely isolated VPN can use arbitrary addresses, but a hybrid architecture with valid IP addresses must be employed to provide hosts with access to the global Internet. The question remains: "How can a site provide access to the global Internet without assigning each host a valid IP address?" There are two general solutions.

Known as an *application gateway* approach, the first solution offers hosts access to Internet services without offering IP-level access. Each site has a multi-homed host connected to both the global Internet (with a globally valid IP address) and the internal network (using a private IP address). The multi-homed host runs a set of application programs, known as *application gateways*, that each handle one service. Hosts at the site do not send datagrams to the global Internet. Instead, they send each request to the appropriate application gateway on the multihomed host, which accesses the service on the Internet and then relays the information back across the internal network. For example, Chapter 27 describes an e-mail gateway that can relay e-mail messages between external hosts and internal hosts.

The chief advantage of the application gateway approach lies in its ability to work without changes to the underlying infrastructure or addressing. The chief disadvantage arises from the lack of generality, which can be summarized:

Each application gateway handles only one specific service; multiple gateways are required for multiple services.

Consequently, although they are useful in special circumstances, application gateways do not solve the problem in a general way. Thus, a second solution was invented.

20.6 Network Address Translation (NAT)

A technology has been created that solves the general problem of providing IP-level access between hosts at a site and the rest of the Internet, without requiring each host at the site to have a globally valid IP address. Known as *Network Address Translation (NAT)*, the technology requires a site to have a single connection to the global Internet and at least one globally valid IP address, G . Address G is assigned to a computer (a multi-homed host or a router) that connects the site to the Internet and runs NAT software. Informally, we refer to a computer that runs NAT software as a *NAT box*; all datagrams pass through the NAT box as they travel from the site out to the Internet or from the Internet into the site.

NAT translates the addresses in both outgoing and incoming datagrams by replacing the source address in each outgoing datagram with G and replacing the destination address in each incoming datagram with the private address of the correct host. Thus, from the view of an external host, all datagrams come from the NAT box and all responses return to the NAT box. From the view of internal hosts, the NAT box appears to be a router that can reach the global Internet.

The chief advantage of NAT arises from its combination of generality and transparency. NAT is more general than application gateways because it allows an arbitrary internal host to access an arbitrary service on a computer in the global Internet. NAT is transparent because it allows an internal host to send and receive datagrams using a private (i.e., nonroutable) address.

To summarize:

Network Address Translation technology provides transparent IP-level access to the Internet from a host with a private address.

20.7 NAT Translation Table Creation

Our overview of NAT omits an important detail because it does not specify how NAT knows which internal host should receive a datagram that arrives from the Internet. In fact, NAT maintains a translation table that it uses to perform the mapping. Each entry in the table specifies two items: the IP address of a host on the Internet and the internal IP address of a host at the site. When an incoming datagram arrives from the Internet, NAT looks up the datagram's destination address in the translation table, extracts the corresponding address of an internal host, replaces the datagram's destination address with the host's address, and forwards the datagram across the local network to the host[†].

The NAT translation table must be in place before a datagram arrives from the Internet. Otherwise, NAT has no way to identify the correct internal host to which the datagram should be forwarded. How and when is the table initialized? There are several possibilities:

- *Manual initialization.* A manager configures the translation table manually before any communication occurs.
- *Outgoing datagrams.* The table is built as a side-effect of sending datagrams. When it receives a datagram from an internal host, NAT creates an entry in the translation table to record the address of the host and the address of the destination.
- *Incoming name lookups.* The table is built as a side-effect of handling domain name lookups. When a host on the Internet looks up the domain name of an internal host to find its IP address[‡], the domain name software creates an entry in the NAT translation table, and then answers the request by sending address *G*. Thus, from outside the site, it appears that all host names at the site map to address *G*.

Each initialization technique has advantages and disadvantages. Manual initialization provides permanent mappings and allows IP datagrams to be sent in either direction at any time. Using an outgoing datagram to initialize the table has the advantage of being automatic, but does not allow communication to be initiated from the outside. Using incoming domain name lookups requires modifying domain name software. It accommodates communication initiated from outside the site, but only works if the sender performs a domain name lookup before sending datagrams.

Most implementations of NAT use outgoing datagrams to initialize the table; the strategy is especially popular among ISPs. To understand why, consider a small ISP that serves dialup customers. Figure 20.5 illustrates the architecture.

[†]Of course, whenever it replaces an address in a datagram header, NAT must recompute the header checksum.

[‡]Chapter 24 describes how the *Domain Name System (DNS)* operates.

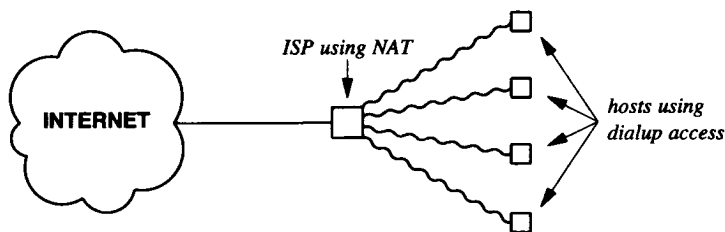


Figure 20.5 The use of NAT by a small ISP that serves dialup customers. NAT translation allows the ISP to assign a private address to each dialup customer.

The ISP must assign an IP address to a customer whenever the customer dials in. NAT permits the ISP to assign private addresses (e.g., the first customer is assigned *10.0.0.1*, the second *10.0.0.2*, and so on). When a customer sends a datagram to a destination on the Internet, NAT uses the outgoing datagram to initialize its translation table.

20.8 Multi-Address NAT

So far, we have described a simplistic implementation of NAT that performs a 1-to-1 address mapping between an external address and an internal address. That is, a 1-to-1 mapping permits at most one computer at the site to access a given machine on the global Internet at any time. In practice, more complex forms of NAT are used that allow multiple hosts at a site to access a given external address concurrently.

One variation of NAT permits concurrency by retaining the 1-to-1 mapping, but allowing the NAT box to hold multiple Internet addresses. Known as *multi-address NAT*, the scheme assigns the NAT box a set of K globally valid addresses, $G_1, G_2, \dots, G_K$. When the first internal host accesses a given destination, the NAT box chooses address G_1 , adds an entry to the translation table, and sends the datagram. If another host initiates contact with the same destination, the NAT box chooses address G_2 , and so on. Thus, multi-address NAT allows up to K internal hosts to access a given destination concurrently.

20.9 Port-Mapped NAT

Another popular variant of NAT provides concurrency by translating TCP or UDP protocol port numbers as well as addresses. Sometimes called *Network Address Port Translation (NAPT)*, the scheme expands the NAT translation table to include additional fields. Besides a pair of source and destination IP addresses, the table contains a pair of source and destination protocol port numbers and a protocol port number used by the NAT box. Figure 20.6 illustrates the contents of the table.

Private Address	Private Port	External Address	External Port	NAT Port	Protocol Used
10.0.0.5	21023	128.10.19.20	80	14003	tcp
10.0.0.1	386	128.10.19.20	80	14010	tcp
10.0.2.6	26600	207.200.75.200	21	14012	tcp
10.0.0.3	1274	128.210.1.5	80	14007	tcp

Figure 20.6 An example of a translation table used by NATP. The table includes port numbers as well as IP addresses.

The table in the figure has entries for four internal computers that are currently accessing destinations on the global Internet. All communication is using TCP. Interestingly, the table shows two internal hosts, *10.0.0.5* and *10.0.0.1*, both accessing protocol port *80* (a Web server) on computer *128.10.19.20*. In this case, it happens that the two source ports being used for the two connections differ. However, source port uniqueness cannot be guaranteed — it could turn out that two internal hosts happen to choose the same source port number. Thus, to avoid potential conflicts, NAT assigns a unique port number to each communication that is used on the Internet. Recall that TCP identifies each connection with a 4-tuple that represents the IP address and protocol port number of each endpoint. The first two items in the table correspond to TCP connections that the two internal hosts identify with the 4-tuples:

$$\begin{aligned} & (10.0.0.5, 23023, 128.10.19.20, 80) \\ & (10.0.0.1, 386, 128.10.19.20, 80) \end{aligned}$$

However, the computer in the Internet that receives datagrams after NATP performs the translation identifies the same two connections with the 4-tuples:

$$\begin{aligned} & (G, 14003, 128.10.19.20, 80) \\ & (G, 14010, 128.10.19.20, 80) \end{aligned}$$

where *G* is the globally valid address of the NAT box.

The primary advantage of NATP lies in the generality it achieves with a single globally valid IP address; the primary disadvantage arises because it restricts communication to TCP or UDP. As long as all communication uses TCP or UDP, NATP allows an internal computer to access multiple external computers, and multiple internal computers to access the same external computer without interference. A port space of 16 bits allows up to 2^{16} pairs of applications to communicate at the same time. To summarize:

Several variants of NAT exist, including the popular NATP form that translates protocol port numbers as well as IP addresses.

20.10 Interaction Between NAT And ICMP

Even straightforward changes to an IP address can cause unexpected side-effects in higher layer protocols. In particular, to maintain the illusion of transparency, NAT must handle ICMP. For example, suppose an internal host uses *ping* to test reachability of a destination on the Internet. The host expects to receive an ICMP *echo reply* for each ICMP *echo request* message it sends. Thus, NAT must forward incoming echo replies to the correct host. However, NAT does not forward all ICMP messages that arrive from the Internet. If routes in the NAT box are incorrect, for example, an ICMP *redirect* message must be processed locally. Thus, when an ICMP message arrives from the Internet, NAT must first determine whether the message should be handled locally or sent to an internal host. Before forwarding to an internal host, NAT translates the ICMP message.

To understand the need for ICMP translation, consider an ICMP *destination unreachable* message. The message contains the header from a datagram, *D*, that caused the error. Unfortunately, NAT translated addresses before sending *D*, so the source address is not the address the internal host used. Thus, before forwarding the message, NAT must open the ICMP message and translate the addresses in *D* so they appear in exactly the form that the internal host used. After making the change, NAT must recompute the checksum in *D*, the checksum in the ICMP header, and the checksum in the outer datagram header.

20.11 Interaction Between NAT And Applications

Although ICMP makes NAT complex, application protocols have a more serious effect. In general, NAT will not work with any application that sends IP addresses or protocol ports as data. For example, when two programs use the *File Transfer Protocol (FTP)* described in Chapter 26, they have a TCP connection between them. As part of the protocol, one program obtains a protocol port on the local machine, converts the number to ASCII, and sends the result across a TCP connection to another program. If the connection between the programs passes through NAT from an internal host to a host on the Internet, the port number in the data stream must be changed to agree with the port number NAT has selected instead of the port the internal host is using. In fact, if NAT fails to open the data stream and change the number, the protocol will fail. Implementations of NAT have been created that recognize popular protocols such as FTP and make the necessary change in the data stream. However, there exist applications that cannot use NAT. To summarize:

NAT affects ICMP and higher layer protocols; except for a few standard applications like FTP, an application protocol that passes IP addresses or protocol port numbers as data will not operate correctly across NAT.

Changing items in a data stream increases the complexity of NAPT in two ways. First, it means that NAPT must have detailed knowledge of each application that transfers such information. Second, if the port numbers are represented in ASCII, as is the case with FTP, changing the value can change the number of octets transferred. Inserting even one additional octet into a TCP connection is difficult because each octet in the stream has a sequence number. Because a sender does not know that additional data has been inserted, it continues to assign sequence numbers without the additional data. When it receives additional data, the receiver will generate acknowledgements that account for the data. Thus, after it inserts additional data, NAT must translate the sequence numbers in each outgoing segment and each incoming acknowledgement.

20.12 Conceptual Address Domains

We have described NAT as a technology that can be used to connect a private network to the global Internet. In fact, NAT can be used to interconnect any two *address domains*. Thus, NAT can be used between two corporations that each have a private network using address 10.0.0.0. More important, NAT can be used at two levels: between a customer's private and an ISP's private address domains as well as between the ISP's address domain and the global Internet. Finally, NAT can be combined with VPN technology to form a hybrid architecture in which private addresses are used within the organization, and NAT is used to provide connectivity between each site and the global Internet.

As an example of multiple levels of NAT, consider an individual who works at home from several computers which are connected to a LAN. The individual can assign private addresses to the computers at home, and use NAT between the home network and the corporate intranet. The corporation can also assign private addresses and use NAT between its intranet and the global Internet.

20.13 Slirp And Masquerade

Two implementations of Network Address Translation have become especially popular; both were designed for the Unix operating system. The *slirp* program, derived from 4.4 BSD, comes with program source code. It was designed for use in a dialup architecture like the one shown in Figure 20.5. Slirp combines PPP and NAT into a single program. It runs on a computer that has: a valid IP address, a permanent Internet connection, and one or more dialup modems. The chief advantage of slirp is that it can use an ordinary user account on a Unix system for general-purpose Internet access. A computer that has a private address dials in and runs slirp. Once slirp begins, the dialup line switches from ASCII commands to PPP. The dialup computer starts PPP and obtains access to the Internet (e.g., to access a Web site).

Slirp implements NAPT — it uses protocol port numbers to demultiplex connections, and can rewrite protocol port numbers as well as IP addresses. It is possible to

have multiple computers (e.g., computers on a LAN) accessing the Internet at the same time through a single occurrence of *slirp* running on a UNIX system.

Another popular implementation of NAT has been designed for the Linux operating system. Known as *masquerade*, the program implements NAPT. Unlike *slirp*, *masquerade* does not require computers to access it via dialup, nor does *masquerade* need a user to login to the UNIX system before starting it. Instead, *masquerade* offers many options; it can be configured to operate like a router between two networks, and it handles most of the NAT variations discussed in this chapter, including the use of multiple IP addresses.

20.14 Summary

Although a private network guarantees privacy, the cost can be high. Virtual Private Network (VPN) technology offers a lower cost alternative that allows an organization to use the global Internet to interconnect multiple sites and use encryption to guarantee that intersite traffic remains private. Like a traditional private network, a VPN can either be completely isolated (in which case hosts are assigned private addresses) or a hybrid architecture that allows hosts to communicate with destinations on the global Internet.

Two technologies exist that provide communication between hosts in different address domains: application gateways and Network Address Translation (NAT). An application gateway acts like a proxy by receiving a request from a host in one domain, sending the request to a destination in another, and then returning the result to the original host. A separate application gateway must be installed for each service.

Network Address Translation provides transparent IP-level access to the global Internet from a host that has a private address. NAT is especially popular among ISPs because it allows customers to access arbitrary Internet services while using a private IP address. Applications that pass address or port information in the data stream will not work with NAT until NAT has been programmed to recognize the application and make the necessary changes in the data; most implementations of NAT only recognize a few (standard) services.

FOR FURTHER STUDY

Many router and software vendors sell Virtual Private Network technologies, usually with a choice of encryption schemes and addressing architecture. Consult the vendors' literature for more information.

Several versions of NAT are also available commercially. The charter of the IETF working group on NAT can be found at:

<http://www.ietf.org/html.charters/nat-charter.html>

In addition, Srisuresh and Holdrege [RFC 2663] defines NAT terminology, and the Internet Draft repository at

<http://www.ietf.org/ID.html>

contains several Internet Drafts on NAT.

More details about the masquerade program can be found in the Linux documentation. A resource page can be found at URL:

<http://ipmasq.cjb.net>

More information on slirp can be found in the program documentation; a resource page for slirp can be found at:

<http://blitzen.canberra.edu.au/slirp>

EXERCISES

- 20.1 Under what circumstances will a VPN transfer substantially more packets than conventional IP when sending the same data across the Internet? Hint: think about encapsulation.
- 20.2 Read the slirp document to find out about *port redirection*. Why is it needed?
- 20.3 What are the potential problems when three address domains are connected by two NAT boxes?
- 20.4 In the previous question, how many times will a destination address be translated? A source address?
- 20.5 Consider an ICMP host unreachable message sent through two NAT boxes that interconnect three address domains. How many address translations will occur? How many translations of protocol port numbers will occur?
- 20.6 Imagine that we decide to create a new Internet parallel to the existing Internet that allocates addresses from the same address space. Can NAT technology be used to connect the two arbitrarily large Internets that use the same address space? If so, explain how. If not, explain why not.
- 20.7 Is NAT completely transparent to a host? To answer the question, try to find a sequence of packets that a host can transmit to determine whether it is located behind a NAT box.
- 20.8 What are the advantages of combining NAT technology with VPN technology? The disadvantages?
- 20.9 Obtain a copy of slirp and instrument it to measure performance. Does slirp processing overhead ever delay datagrams? Why or why not?
- 20.10 Obtain NAT and configure it on a Linux system between a private address domain and the Internet. Which well-known services work correctly and which do not?
- 20.11 Read about a variant of NAT called *twice NAT* that allows communication to be initiated from either side of the NAT box at any time. How does twice NAT ensure that translations are consistent? If two instances of twice NAT are used to interconnect three address domains, is the result completely transparent to all hosts?

Client-Server Model Of Interaction

21.1 Introduction

Early chapters present the details of TCP/IP technology, including the protocols that provide basic services and the router architecture that provides needed routing information. Now that we understand the basic technology, we can examine application programs that profit from the cooperative use of a TCP/IP internet. While the example applications are both practical and interesting, they do not comprise the main emphasis. Instead, focus rests on the patterns of interaction among the communicating application programs. The primary pattern of interaction among cooperating applications is known as the *client-server* paradigm[†]. Client-server interaction forms the basis of most network communication, and is fundamental because it helps us understand the foundation on which distributed algorithms are built. This chapter considers the relationship between client and server; later chapters illustrate the client-server pattern with further examples.

21.2 The Client-Server Model

The term *server* applies to any program that offers a service that can be reached over a network. A server accepts a request over the network, performs its service, and returns the result to the requester. For the simplest services, each request arrives in a single IP datagram and the server returns a response in another datagram.

[†]Marketing literature sometimes substitutes the term *application-server* for client-server; the underlying scientific principle is unchanged.

An executing program becomes a *client* when it sends a request to a server and waits for a response. Because the client-server model is a convenient and natural extension of interprocess communication on a single machine, it is easy to build programs that use the model to interact.

Servers can perform simple or complex tasks. For example, a *time-of-day server* merely returns the current time whenever a client sends the server a packet. A *web server* receives requests from a browser to fetch a copy of a Web page; the server obtains a copy of the file for the page and returns it to the browser.

Usually, servers are implemented as application programs†. The advantage of implementing servers as application programs is that they can execute on any computing system that supports TCP/IP communication. Thus, the server for a particular service can execute on a timesharing system along with other programs, or it can execute on a personal computer. Multiple servers can offer the same service, and can execute on the same machine or on multiple machines. In fact, managers commonly replicate copies of a given server onto physically independent machines to increase reliability or improve performance. If a computer's primary purpose is support of a particular server program, the term "server" may be applied to the computer as well as to the server program. Thus, one hears statements such as "machine A is our file server."

21.3 A Simple Example: UDP Echo Server

The simplest form of client-server interaction uses unreliable datagram delivery to convey messages from a client to a server and back. Consider, for example, a *UDP echo server*. The mechanics are straightforward as Figure 21.1 shows. At the server site, a *UDP echo server process* begins by negotiating with its operating system for permission to use the UDP port ID reserved for the *echo* service, the *UDP echo port*. Once it has obtained permission, the echo server process enters an infinite loop that has three steps: (1) wait for a datagram to arrive at the echo port, (2) reverse the source and destination addresses‡ (including source and destination IP addresses as well as UDP port ids), and (3) return the datagram to its original sender. At some other site, a program becomes a *UDP echo client* when it allocates an unused UDP protocol port, sends a UDP message to the UDP echo server, and awaits the reply. The client expects to receive back exactly the same data as it sent.

The UDP echo service illustrates two important points that are generally true about client-server interaction. The first concerns the difference between the lifetime of servers and clients:

A server starts execution before interaction begins and (usually) continues to accept requests and send responses without ever terminating. A client is any program that makes a request and awaits a response; it (usually) terminates after using a server a finite number of times.

†Many operating systems refer to a running application program as a *process*, a *user process*, or a *task*.

‡One of the exercises suggests considering this step in more detail.

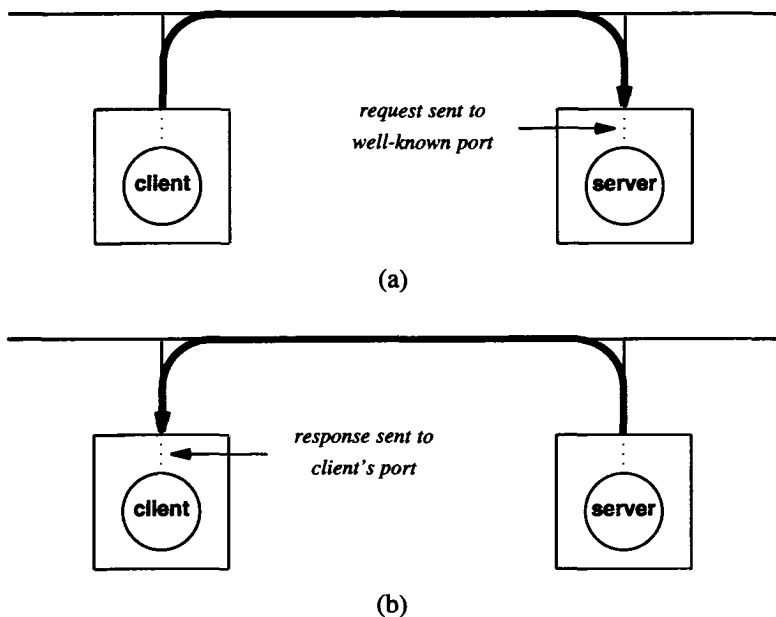


Figure 21.1 UDP echo as an example of the client-server model. In (a) the client sends a request to the server at a known IP address and at a well-known UDP port, and in (b) the server returns a response. Clients use any UDP port that is available.

The second point, which is more technical, concerns the use of reserved and non-reserved port identifiers:

A server waits for requests at a well-known port that has been reserved for the service it offers. A client allocates an arbitrary, unused, nonreserved port for its communication.

In a client-server interaction, only one of the two ports needs to be reserved. Assigning a unique port identifier to each service makes it easy to build both clients and servers.

Who would use an echo service? It is not a service that the average user finds interesting. However, programmers who design, implement, measure, or modify network protocol software, or network managers who test routes and debug communication problems, often use echo servers in testing. For example, an echo service can be used to determine if it is possible to reach a remote machine.

21.4 Time And Date Service

The echo server is extremely simple, and little code is required to implement either the server or client side (provided that the operating system offers a reasonable way to access the underlying UDP/IP protocols). Our second example, a time server, shows that even simple client-server interaction can provide useful services. The problem a time server solves is that of setting a computer's time-of-day clock. The time of day clock is a hardware device that maintains the current date and time, making it available to programs. Once set, the time of day clock keeps time as accurately as a wristwatch.

Some systems solve the problem by asking a programmer to type in the time and date when the system boots. The system increments the clock periodically (e.g., every second). When an application program asks for the date or time, the system consults the internal clock and formats the time of day in human readable form. Client-server interaction can be used to set the system clock automatically when a machine boots. To do so, a manager configures one machine, typically the machine with the most accurate clock, to run a time-of-day server. When other machines boot, they contact the server to obtain the current time.

21.4.1 Representation for the Date and Time

How should an operating system maintain the date and time-of-day? One useful representation stores the time and date as the count of seconds since an epoch date. For example, the UNIX operating system uses the zeroth second of January 1, 1970 as its epoch date. The TCP/IP protocols also define an epoch date and report times as seconds past the epoch. For TCP/IP, the epoch is defined to be the zeroth second of January 1, 1900 and the time is kept in a 32-bit integer, a representation that accommodates all dates in the near future.

Keeping the date as the time in seconds since an epoch makes the representation compact and allows easy comparison. It ties together the date and time of day and makes it possible to measure time by incrementing a single binary integer.

21.4.2 Local and Universal Time

Given an epoch date and representation for the time, to what time zone does the count refer? When two systems communicate across large geographic distances, using the local time zone from one or the other becomes difficult; they must agree on a standard time zone to keep values for date and time comparable. Thus, in addition to defining a representation for the date and choosing an epoch, the TCP/IP time server standard specifies that all values are given with respect to a single time zone. Originally called Greenwich Mean Time, the time zone is now known as *universal coordinated time* or *universal time*.

The interaction between a client and a server that offers time service works much like an echo server. At the server side, the server application obtains permission to use the reserved port assigned to time servers, waits for a UDP message directed to that port, and responds by sending a UDP message that contains the current time in a 32-bit integer. We can summarize:

Sending a datagram to a time server is equivalent to making a request for the current time; the server responds by returning a UDP message that contains the current time.

21.5 The Complexity of Servers

In our examples so far, servers are fairly simple because they are sequential. That is, the server processes one request at a time. After accepting a request, the server forms a reply and sends it before going back to see if another request has arrived. We implicitly assume that the operating system will queue requests that arrive for a server while it is busy, and that the queue will not become too long because the server has only a trivial amount of work to do.

In practice, servers are usually much more difficult to build than clients because they need to accommodate multiple concurrent requests, even if a single request takes considerable time to process. For example, consider a file transfer server responsible for copying a file to another machine on request. Typically, servers have two parts: a single master program that is responsible for accepting new requests, and a set of slaves that are responsible for handling individual requests. The master server performs the following five steps:

Open port

The master opens the well-known port at which it can be reached.

Wait for client

The master waits for a new client to send a request.

Choose port

If necessary, the master allocates a new local protocol port for this request and informs the client (we will see that this step is unnecessary with TCP and most uses of UDP).

Start Slave

The master starts an independent, concurrent slave to handle this request (e.g., in UNIX, it forks a copy of the server process). Note that the slave handles one request and then terminates — the slave does not wait for requests from other clients.

Continue

The master returns to the *wait* step and continues accepting new requests while the newly created slave handles the previous request concurrently.

Because the master starts a slave for each new request, processing proceeds concurrently. Thus, requests that require little time to complete can finish earlier than requests that take longer, independent of the order in which they are started. For example, suppose the first client that contacts a file server requests a large file transfer that

takes many minutes. If a second client contacts the server to request a transfer that takes only a few seconds, the second transfer can start and complete while the first transfer proceeds.

In addition to the complexity that results because servers handle concurrent requests, complexity also arises because servers must enforce authorization and protection rules. Server programs usually need to execute with highest privilege because they must read system files, keep logs, and access protected data. The operating system will not restrict a server program if it attempts to access users' files. Thus, servers cannot blindly honor requests from other sites. Instead, each server takes responsibility for enforcing the system access and protection policies.

Finally, servers must protect themselves against malformed requests or against requests that will cause the server program itself to abort. Often, it is difficult to foresee potential problems. For example, one project at Purdue University designed a file server that allowed student operating systems to access files on a UNIX timesharing system. Students discovered that requesting the server to open a file named `/dev/tty` caused the server to abort because UNIX associates that name with the control terminal to which a program is attached. The server, created at system startup, had no such terminal. Once an abort occurred, no client could access files until a systems programmer restarted the server.

A more serious example of server vulnerability became known in the fall of 1988 when a student at Cornell University built a *worm* program that attacked computers on the global Internet. Once the worm started running on a machine, it searched the Internet for computers with servers that it knew how to exploit, and used the servers to create more copies of itself. In one of the attacks, the worm used a bug in the UNIX *fingerd* server. Because the server did not check incoming requests, the worm was able to send an illegal string of input that caused the server to overwrite parts of its internal data areas. The server, which executed with highest privilege, then misbehaved, allowing the worm to create copies of itself.

We can summarize our discussion of servers:

Servers are usually more difficult to build than clients because, although they can be implemented with application programs, servers must enforce all the access and protection policies of the computer system on which they run, and must protect themselves against all possible errors.

21.6 RARP Server

So far, all our examples of client-server interaction require the client to know the complete server address. The RARP protocol from Chapter 6 provides an example of client-server interaction with a slightly different twist. Recall that a machine can use RARP to find its IP address at startup. Instead of having the client communicate directly with a server, RARP clients broadcast their requests. One or more machines executing RARP server processes respond, each returning a packet that answers the query.

There are two significant differences between a RARP server and a UDP echo or time server. First, RARP packets travel across the physical network directly in hardware frames, not in IP datagrams. Thus, unlike the UDP echo server which allows a client to contact a server anywhere on an internet, the RARP server requires the client to be on the same physical network. Second, RARP cannot be implemented by an application program. Echo and time servers can be built as application programs because they use UDP. By contrast, a RARP server needs access to raw hardware packets.

21.7 Alternatives To The Client-Server Model

What are the alternatives to client-server interaction, and when might they be attractive? This section gives an answer to these questions.

In the client-server model, programs usually act as clients when they need information, but it is sometimes important to minimize such interactions. The ARP protocol from Chapter 5 gives one example. It uses a modified form of client-server interaction to obtain physical address mappings. Machines that use ARP keep a cache of answers to improve the efficiency of later queries. Caching improves the performance of client-server interaction in cases where the recent history of queries is a good indicator of future use.

Although caching improves performance, it does not change the essence of client-server interaction. The essence lies in our assumption that processing must be driven by demand. We have assumed that a program executes until it needs information and then acts as a client to obtain the needed information. Taking a demand-driven view of the world is natural and arises from experience. Caching helps alleviate the cost of obtaining information by lowering the retrieval cost for all except the first process that makes a request.

How can we lower the cost of information retrieval for the first request? In a distributed system, it may be possible to have concurrent background activities that collect and propagate information *before* any particular program requests it, making retrieval costs low even for the initial request. More important, precollecting information can allow a given system to continue executing even though other machines or the networks connecting them fail.

Precollection is the basis for the 4BSD UNIX *ruptime* command. When invoked, *ruptime* reports the CPU load and time since system startup for each machine on the local network. A background program running on each machine uses UDP to broadcast information about the machine periodically. The same program also collects incoming information and places it in a file. Because machines propagate information continuously, each machine has a copy of the latest information on hand; a client seeking information never needs to access the network. Instead, it reads the information from secondary storage and prints it in a readable form.

The chief advantage of having information collected locally before the client needs it is speed. The *ruptime* command responds immediately when invoked without waiting for messages to traverse the network. A second benefit occurs because the client can

find out something about machines that are no longer operating. In particular, if a machine stops broadcasting information, the client can report the time elapsed since the last broadcast (i.e., it can report how long the machine has been off-line).

Precollection has one major disadvantage: it uses processor time and network bandwidth even when no one cares about the data being collected. For example, the runtime broadcast and collection continues running throughout the night, even if no one is logged in to read the information. If only a few machines connect to a given network, precollection cost is insignificant. It can be thought of as an innocuous background activity. For networks with many hosts, however, the large volume of broadcast traffic generated by precollection makes it too expensive. In particular, the cost of reading and processing broadcast messages becomes high. Thus, precollection is not among the most popular alternatives to client-server.

21.8 Summary

Distributed programs require network communication. Such programs often fall into a pattern of use known as client-server interaction. A server process awaits a request and performs action based on the request. The action usually includes sending a response. A client program formulates a request, sends it to a server, and then awaits a reply.

We have seen examples of clients and servers and found that some clients send requests directly, while others broadcast requests. Broadcast is especially useful on a local network when a machine does not know the address of a server.

We also noted that if servers use internet protocols like UDP, they can accept and respond to requests across an internet. If they communicate using physical frames and physical hardware addresses, they are restricted to a single physical network.

Finally, we considered an alternative to the client-server paradigm that uses precollection of information to avoid delays. An example of precollection came from a machine status service.

FOR FURTHER STUDY

UDP echo service is defined in Postel [RFC 862]. The *UNIX Programmer's Manual* describes the *runtime* command (also see the related description of *rwho*). Feinler *et. al.* [1985] specifies many standard server protocols not discussed here, including discard, character generation, day and time, active users, and quote of the day. The next chapters consider others.

EXERCISES

- 21.1** Build a UDP echo client that sends a datagram to a specified echo server, awaits a reply, and compares it to the original message.
- 21.2** Carefully consider the manipulation of IP addresses in a UDP echo server. Under what conditions is it incorrect to create new IP addresses by reversing the source and destination IP addresses?
- 21.3** As we have seen, servers can be implemented by separate application programs or by building server code into the protocol software in an operating system. What are the advantages and disadvantages of having an application program (user process) per server?
- 21.4** Suppose you do not know the IP address of a local machine running a UDP echo server, but you know that it responds to requests sent to port 7. Is there an IP address you can use to reach it?
- 21.5** Build a client for the UDP time service.
- 21.6** Characterize situations in which a server can be located on a separate physical network from its client. Can a RARP server ever be located on a separate physical network from its clients? Why or why not?
- 21.7** What is the chief disadvantage of having all machines broadcast their status periodically?
- 21.8** Examine the format of data broadcast by the servers that implement the 4BSD UNIX *ruptime* command. What information is available to the client in addition to machine status?
- 21.9** What servers are running on computers at your site? If you do not have access to system configuration files that list the servers started for a given computer, see if your system has a command that prints a list of open TCP and UDP ports (e.g., the UNIX *netstat* command).
- 21.10** Some servers allow a manager to gracefully shut them down or restart them. What is the advantage of graceful shutdown?

The Socket Interface

22.1 Introduction

So far, we have concentrated on discussing the principles and concepts that underlie the TCP/IP protocols without specifying the interface between the application programs and the protocol software. This chapter reviews one example of an *Application Program Interface (API)*, the interface between application programs and TCP/IP protocols. There are two reasons for postponing the discussion of APIs. First, in principle we must distinguish between the interface and TCP/IP protocols because the standards do not specify exactly how application programs interact with protocol software. Thus, the interface architecture is not standardized; its design lies outside the scope of the protocol suite. Second, in practice, it is inappropriate to tie the protocols to a particular API because no single interface architecture works well on all systems. In particular, because protocol software resides in a computer's operating system, interface details depend on the operating system.

Despite the lack of a standard, reviewing an example will help us understand how programmers use TCP/IP. Although the example we have chosen is from the BSD UNIX operating system, it has become, de facto, a standard that is widely accepted and used in many systems. In particular, it forms the basis for Microsoft's *Windows Sockets*[†] interface. The reader should keep in mind that our goal is merely to give one concrete example, not to prescribe how APIs should be designed. The reader should also remember that the operations listed here are not part of the TCP/IP standards.

[†]Programmers often use the term *WINSOCK* as a replacement for Windows Sockets.

22.2 The UNIX I/O Paradigm And Network I/O

Developed in the late 1960s and early 1970s, UNIX was originally designed as a timesharing system for single processor computers. It is a process-oriented operating system in which each application program executes as a user level process. An application program interacts with the operating system by making *system calls*. From the programmer's point of view, system calls look and behave exactly like other procedure calls. They take arguments and return one or more results. Arguments can be values (e.g., an integer count) or pointers to objects in the application program (e.g., a buffer to be filled with characters).

Derived from those in Multics and earlier systems, the UNIX input and output (I/O) primitives follow a paradigm sometimes referred to as *open-read-write-close*. Before a user process can perform I/O operations, it calls *open* to specify the file or device to be used and obtains permission. The call to *open* returns a small integer *file descriptor*[†] that the process uses when performing I/O operations on the opened file or device. Once an object has been opened, the user process makes one or more calls to *read* or *write* to transfer data. *Read* transfers data into the user process; *write* transfers data from the user process to the file or device. Both *read* and *write* take three arguments that specify the file descriptor to use, the address of a buffer, and the number of bytes to transfer. After all transfer operations are complete, the user process calls *close* to inform the operating system that it has finished using the object (the operating system automatically closes all open descriptors if a process terminates without calling *close*).

22.3 Adding Network I/O to UNIX

Originally, UNIX designers cast all I/O operations in the open-read-write-close paradigm described above. The scheme included I/O for character-oriented devices like keyboards and block-oriented devices like disks and data files. An early implementation of TCP/IP under UNIX also used the open-read-write-close paradigm with a special file name, */dev/tcp*.

The group adding network protocols to BSD UNIX decided that because network protocols are more complex than conventional I/O devices, interaction between user processes and network protocols must be more complex than interactions between user processes and conventional I/O facilities. In particular, the protocol interface must allow programmers to create both server code that awaits connections passively as well as client code that forms connections actively. Furthermore, application programs sending datagrams may wish to specify the destination address along with each datagram instead of binding destinations at the time they call *open*. To handle all these cases, the designers chose to abandon the traditional UNIX open-read-write-close paradigm, and added several new operating system calls as well as new library routines. Adding network protocols to UNIX increased the complexity of the I/O interface substantially.

Further complexity arises in the UNIX protocol interface because designers attempted to build a general mechanism to accommodate many protocols. For example,

[†]The term "file descriptor" arises because in UNIX all devices are mapped into the file system name space. In most cases, I/O operations on files and devices are indistinguishable.

the generality makes it possible for the operating system to include software for other protocol suites as well as TCP/IP, and to allow an application program to use one or more of them at a time. As a consequence, the application program cannot merely supply a 32-bit address and expect the operating system to interpret it correctly. The application must explicitly specify that the 32-bit number represents an IP address.

22.4 The Socket Abstraction

The basis for network I/O in the socket API centers on an abstraction known as the *socket*†. We think of a socket as a generalization of the UNIX file access mechanism that provides an endpoint for communication. As with file access, application programs request the operating system to create a socket when one is needed. The system returns a small integer that the application program uses to reference the newly created socket. The chief difference between file descriptors and socket descriptors is that the operating system binds a file descriptor to a specific file or device when the application calls *open*, but it can create sockets without binding them to specific destination addresses. The application can choose to supply a destination address each time it uses the socket (e.g., when sending datagrams), or it can choose to bind the destination address to the socket and avoid specifying the destination repeatedly (e.g., when making a TCP connection).

Whenever it makes sense, sockets perform exactly like UNIX files or devices, so they can be used with traditional operations like *read* and *write*. For example, once an application program creates a socket and creates a TCP connection from the socket to a foreign destination, the program can use *write* to send a stream of data across the connection (the application program at the other end can use *read* to receive it). To make it possible to use primitives like *read* and *write* with both files and sockets, the operating system allocates socket descriptors and file descriptors from the same set of integers and makes sure that if a given integer has been allocated as a file descriptor, it will not also be allocated as a socket descriptor.

22.5 Creating A Socket

The *socket* function creates sockets on demand. It takes three integer arguments and returns an integer result:

```
result = socket(pf, type, protocol)
```

Argument *pf* specifies the protocol family to be used with the socket. That is, it specifies how to interpret addresses when they are supplied. Current families include the TCP/IP internet (PF_INET), Xerox Corporation PUP internet (PF_PUP), Apple Computer Incorporated AppleTalk network (PF_APPLETALK), and UNIX file system (PF_UNIX) as well as many others‡.

†For now, we will describe sockets as part of the operating system as they are implemented in UNIX; later sections describe how other operating systems use library routines to provide a socket API.

‡In UNIX, application programs contain symbolic names like *PF_INET*; system files contain the definitions that specify numeric values for each name.

Argument *type* specifies the type of communication desired. Possible types include reliable stream delivery service (SOCK_STREAM) and connectionless datagram delivery service (SOCK_DGRAM), as well as a raw type (SOCK_RAW) that allows privileged programs to access low-level protocols or network interfaces. Two additional types were planned, but not implemented.

Although the general approach of separating protocol families and types may seem sufficient to handle all cases easily, it does not. First, it may be that a given family of protocols does not support one or more of the possible service types. For example, the UNIX family has an interprocess communication mechanism called a *pipe* that uses a reliable stream delivery service, but has no mechanism for sequenced packet delivery. Thus, not all combinations of protocol family and service type make sense. Second, some protocol families have multiple protocols that support one type of service. For example, it may be that a single protocol family has two connectionless datagram delivery services. To accommodate multiple protocols within a family, the *socket* call has a third argument that can be used to select a specific protocol. To use the third argument, the programmer must understand the protocol family well enough to know the type of service each protocol supplies.

Because the designers tried to capture many of the conventional UNIX operations in their socket design, they needed a way to simulate the UNIX pipe mechanism. It is not necessary to understand the details of pipes; only one salient feature is important: pipes differ from standard network operations because the calling process creates both endpoints for the communication simultaneously. To accommodate pipes, the designers added a *socketpair* function that takes the form:

```
socketpair(pf, type, protocol, sarray)
```

Socketpair has one more argument than the *socket* procedure, *sarray*. The additional argument gives the address of a two-element integer array. *Socketpair* creates two sockets simultaneously and places the two socket descriptors in the two elements of *sarray*. Readers should understand that *socketpair* is not meaningful when applied to the TCP/IP protocol family (it has been included here merely to make our description of the interface complete).

22.6 Socket Inheritance And Termination

UNIX uses the *fork* and *exec* system calls to start new application programs. It is a two-step procedure. In the first step, *fork* creates a separate copy of the currently executing application program. In the second step, the new copy replaces itself with the desired application program. When a program calls *fork*, the newly created copy inherits access to all open sockets just as it inherits access to all open files. When a program calls *exec*, the new application retains access to all open sockets. We will see that master servers use socket inheritance when they create slave servers to handle a specific connection. Internally, the operating system keeps a reference count associated with each socket, so it knows how many application programs (processes) have access to it.

Both the old and new processes have the same access rights to existing sockets, and both can access the sockets. Thus, it is the responsibility of the programmer to ensure that the two processes use the shared socket meaningfully.

When a process finishes using a socket it calls *close*. *Close* has the form:

```
close(socket)
```

where argument *socket* specifies the descriptor of a socket to close. When a process terminates for any reason, the system closes all sockets that remain open. Internally, a call to *close* decrements the reference count for a socket and destroys the socket if the count reaches zero.

22.7 Specifying A Local Address

Initially, a socket is created without any association to local or destination addresses. For the TCP/IP protocols, this means no local protocol port number has been assigned and no destination port or IP address has been specified. In many cases, application programs do not care about the local address they use and are willing to allow the protocol software to choose one for them. However, server processes that operate at a well-known port must be able to specify that port to the system. Once a socket has been created, a server uses the *bind* function to establish a local address for it. *Bind* has the following form:

```
bind(socket, localaddr, addrlen)
```

Argument *socket* is the integer descriptor of the socket to be bound. Argument *localaddr* is a structure that specifies the local address to which the socket should be bound, and argument *addrlen* is an integer that specifies the length of the address measured in bytes. Instead of giving the address merely as a sequence of bytes, the designers chose to use a structure for addresses as Figure 22.1 illustrates.

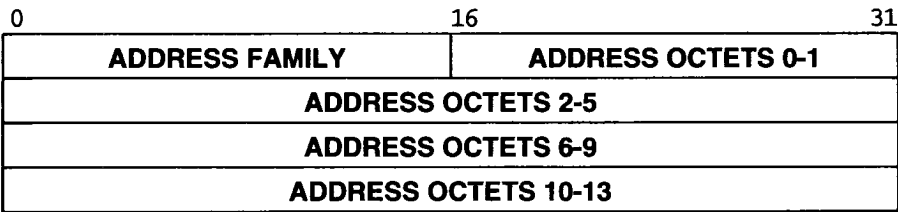


Figure 22.1 The *sockaddr* structure used when passing a TCP/IP address to the socket interface.

The structure, generically named *sockaddr*, begins with a 16-bit *ADDRESS FAMILY* field that identifies the protocol suite to which the address belongs. It is followed by an address of up to 14 octets. When declared in C, the socket address structure is a union of structures for all possible address families.

The value in the *ADDRESS FAMILY* field determines the format of the remaining address octets. For example, the value 2† in the *ADDRESS FAMILY* field means the remaining address octets contain a TCP/IP address. Each protocol family defines how it will use octets in the address field. For TCP/IP addresses, the socket address is known as *sockaddr_in*. It includes both an IP address and a protocol port number (i.e., an internet socket address structure can contain both an IP address and a protocol port at that address). Figure 22.2 shows the exact format of a TCP/IP socket address.

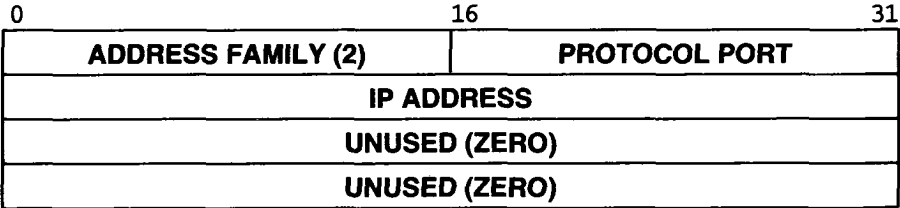


Figure 22.2 The format of a socket address structure (*sockaddr_in*) when used with a TCP/IP address. The structure includes both an IP address and a protocol port at that address.

Although it is possible to specify arbitrary values in the address structure when calling *bind*, not all possible bindings are valid. For example, the caller might request a local protocol port that is already in use by another program, or it might request an invalid IP address. In such cases, the *bind* call fails and returns an error code.

22.8 Connecting Sockets To Destination Addresses

Initially, a socket is created in the *unconnected state*, which means that the socket is not associated with any foreign destination. The function *connect* binds a permanent destination to a socket, placing it in the *connected state*. An application program must call *connect* to establish a connection before it can transfer data through a reliable stream socket. Sockets used with connectionless datagram services need not be connected before they are used, but doing so makes it possible to transfer data without specifying the destination each time.

The *connect* function has the form:

```
connect(socket, destaddr, addrlen)
```

†UNIX uses the symbolic name *PF_INET* to denote TCP/IP addresses.

Argument *socket* is the integer descriptor of the socket to connect. Argument *destaddr* is a socket address structure that specifies the destination address to which the socket should be bound. Argument *addrlen* specifies the length of the destination address measured in bytes.

The semantics of *connect* depend on the underlying protocols. Selecting the reliable stream delivery service in the PF_INET family means choosing TCP. In such cases, *connect* builds a TCP connection with the destination and returns an error if it cannot. In the case of connectionless service, *connect* does nothing more than store the destination address locally.

22.9 Sending Data Through A Socket

Once an application program has established a socket, it can use the socket to transmit data. There are five possible functions from which to choose: *send*, *sendto*, *sendmsg*, *write*, and *writen*. *Send*, *write*, and *writen* only work with connected sockets because they do not allow the caller to specify a destination address. The differences between the three are minor. *Write* takes three arguments:

```
write(socket, buffer, length)
```

Argument *socket* contains an integer socket descriptor (*write* can also be used with other types of descriptors). Argument *buffer* contains the address of the data to be sent, and argument *length* specifies the number of bytes to send. The call to *write* blocks until the data can be transferred (e.g., it blocks if internal system buffers for the socket are full). Like most system calls, *write* returns an error code to the application calling it, allowing the programmer to know if the operation succeeded.

The system call *writen* works like *write* except that it uses a “gather write” form, making it possible for the application program to write a message without copying the message into contiguous bytes of memory. *Writen* has the form:

```
writen(socket, iovector, vectorlen)
```

Argument *iovector* gives the address of an array of type *iovec* that contains a sequence of pointers to the blocks of bytes that form the message. As Figure 22.3 shows, a *length* accompanies each pointer. Argument *vectorlen* specifies the number of entries in *iovector*.

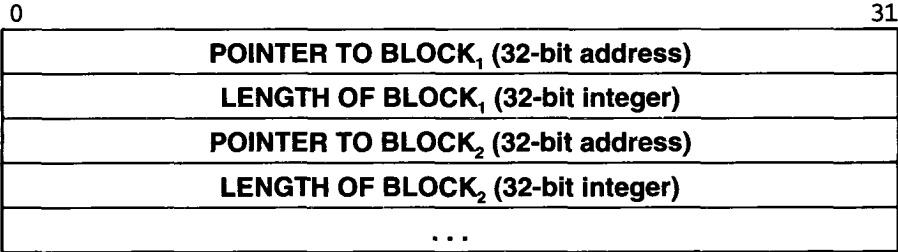


Figure 22.3 The format of an iovector of type *iovec* used with *writenv* and *readv*.

The *send* function has the form:

```
send(socket, message, length, flags)
```

where argument *socket* specifies the socket to use, argument *message* gives the address of the data to be sent, argument *length* specifies the number of bytes to be sent, and argument *flags* controls the transmission. One value for *flags* allows the sender to specify that the message should be sent out-of-band on sockets that support such a notion. For example, recall from Chapter 13 that out-of-band messages correspond to TCP’s notion of urgent data. Another value for *flags* allows the caller to request that the message be sent without using local routing tables. The intention is to allow the caller to take control of routing, making it possible to write network debugging software. Of course, not all sockets support all requests from arbitrary programs. Some requests require the program to have special privileges; others are simply not supported on all sockets.

Functions *sendto* and *sendmsg* allow the caller to send a message through an unconnected socket because they both require the caller to specify a destination. *Sendto*, which takes the destination address as an argument, has the form:

```
sendto(socket, message, length, flags, destaddr, addrlen)
```

The first four arguments are exactly the same as those used with the *send* function. The final two arguments specify a destination address and give the length of that address. Argument *destaddr* specifies the destination address using the *sockaddr_in* structure as defined in Figure 22.2.

A programmer may choose to use function *sendmsg* in cases where the long list of arguments required for *sendto* makes the program inefficient or difficult to read. *Sendmsg* has the form:

```
sendmsg(socket, messagestruct, flags)
```

where argument *messagestruct* is a structure of the form illustrated in Figure 22.4. The structure contains information about the message to be sent, its length, the destination

address, and the address length. This call is especially useful because there is a corresponding input operation (described below) that produces a message structure in exactly the same format.

0	31
POINTER TO SOCKETADDR	
SIZE OF SOCKETADDR	
POINTER TO IOVEC LIST	
LENGTH OF IOVEC LIST	
POINTER TO ACCESS RIGHTS LIST	
LENGTH OF ACCESS RIGHTS LIST	

Figure 22.4 The format of message structure *messagestruct* used by *sendmsg*.

22.10 Receiving Data Through A Socket

Analogous to the five different output operations, the socket API offers five functions that a process can use to receive data through a socket: *read*, *readv*, *recv*, *recvfrom*, and *recvmsg*. The conventional input operation, *read*, can only be used when the socket is connected. It has the form:

```
read(descriptor, buffer, length)
```

where *descriptor* gives the integer descriptor of a socket or file descriptor from which to read data, *buffer* specifies the address in memory at which to store the data, and *length* specifies the maximum number of bytes to read.

An alternative form, *readv*, allows the caller to use a “scatter read” style of interface that places the incoming data in noncontiguous locations. *Readv* has the form:

```
readv(descriptor, iovector, vectorlen)
```

Argument *iovector* gives the address of a structure of type *iovec* (see Figure 22.3) that contains a sequence of pointers to blocks of memory into which the incoming data should be stored. Argument *vectorlen* specifies the number of entries in *iovector*.

In addition to the conventional input operations, there are three additional functions for network message input. Processes call *recv* to receive data from a connected socket. It has the form:

```
recv(socket, buffer, length, flags)
```

Argument *socket* specifies a socket descriptor from which data should be received. Argument *buffer* specifies the address in memory into which the message should be placed, and argument *length* specifies the length of the buffer area. Finally, argument *flags* allows the caller to control the reception. Among the possible values for the *flags* argument is one that allows the caller to look ahead by extracting a copy of the next incoming message without removing the message from the socket.

The function *recvfrom* allows the caller to specify input from an unconnected socket. It includes additional arguments that allow the caller to specify where to record the sender's address. The form is:

```
recvfrom(socket, buffer, length, flags, fromaddr, addrlen)
```

The two additional arguments, *fromaddr* and *addrlen*, are pointers to a socket address structure and an integer. The operating system uses *fromaddr* to record the address of the message sender and uses *fromlen* to record the length of the sender's address. Notice that the output operation *sendto*, discussed above, takes an address in exactly the same form as *recvfrom* generates. Thus, sending replies is easy.

The final function used for input, *recvmsg*, is analogous to the *sendmsg* output operation. *Recvmsg* operates like *recvfrom*, but requires fewer arguments. Its form is:

```
recvmsg(socket, messagestruct, flags)
```

where argument *messagestruct* gives the address of a structure that holds the address for an incoming message as well as locations for the sender's address. The structure produced by *recvmsg* is exactly the same as the structure used by *sendmsg*, making them operate well as a pair.

22.11 Obtaining Local And Remote Socket Addresses

We said that newly created processes inherit the set of open sockets from the process that created them. Sometimes, a newly created process needs to determine the destination address to which a socket connects. A process may also wish to determine the local address of a socket. Two functions provide such information: *getpeername* and *getsockname* (despite their names, both deal with what we think of as "addresses").

A process calls *getpeername* to determine the address of the peer (i.e., the remote end) to which a socket connects. It has the form:

```
getpeername(socket, destaddr, addrlen)
```

Argument *socket* specifies the socket for which the address is desired. Argument *destaddr* is a pointer to a structure of type *sockaddr* (see Figure 22.1) that will receive the socket address. Finally, argument *addrlen* is a pointer to an integer that will receive the length of the address. *Getpeername* only works with connected sockets.

Function *getsockname* returns the local address associated with a socket. It has the form:

```
getsockname(socket, localaddr, addrlen)
```

As expected, argument *socket* specifies the socket for which the local address is desired. Argument *localaddr* is a pointer to a structure of type *sockaddr* that will contain the address, and argument *addrlen* is a pointer to an integer that will contain the length of the address.

22.12 Obtaining And Setting Socket Options

In addition to binding a socket to a local address or connecting it to a destination address, the need arises for a mechanism that permits application programs to control the socket. For example, when using protocols that use timeout and retransmission, the application program may want to obtain or set the timeout parameters. It may also want to control the allocation of buffer space, determine if the socket allows transmission of broadcast, or control processing of out-of-band data. Rather than add new functions for each new control operation, the designers decided to build a single mechanism. The mechanism has two operations: *getsockopt* and *setsockopt*.

Function *getsockopt* allows the application to request information about the socket. A caller specifies the socket, the option of interest, and a location at which to store the requested information. The operating system examines its internal data structures for the socket and passes the requested information to the caller. The call has the form:

```
getsockopt(socket, level, optionid, optionval, length)
```

Argument *socket* specifies the socket for which information is needed. Argument *level* identifies whether the operation applies to the socket itself or to the underlying protocols being used. Argument *optionid* specifies a single option to which the request applies. The pair of arguments *optionval* and *length* specify two pointers. The first gives the address of a buffer into which the system places the requested value, and the second gives the address of an integer into which the system places the length of the option value.

Function *setsockopt* allows an application program to set a socket option using the set of values obtained with *getsockopt*. The caller specifies a socket for which the option should be set, the option to be changed, and a value for the option. The call to *setsockopt* has the form:

```
setsockopt(socket, level, optionid, optionval, length)
```

where the arguments are like those for *getsockopt*, except that the *length* argument contains the length of the option being passed to the system. The caller must supply a legal value for the option as well as a correct length for that value. Of course, not all options

apply to all sockets. The correctness and semantics of individual requests depend on the current state of the socket and the underlying protocols being used.

22.13 Specifying A Queue Length For A Server

One of the options that applies to sockets is used so frequently, a separate function has been dedicated to it. To understand how it arises, consider a server. The server creates a socket, binds it to a well-known protocol port, and waits for requests. If the server uses a reliable stream delivery, or if computing a response takes nontrivial amounts of time, it may happen that a new request arrives before the server finishes responding to an old request. To avoid having protocols reject or discard incoming requests, a server must tell the underlying protocol software that it wishes to have such requests enqueued until it has time to process them.

The function *listen* allows servers to prepare a socket for incoming connections. In terms of the underlying protocols, *listen* puts the socket in a passive mode ready to accept connections. When the server invokes *listen*, it also informs the operating system that the protocol software should enqueue multiple simultaneous requests that arrive at the socket. The form is:

`listen(socket, qlength)`

Argument *socket* gives the descriptor of a socket that should be prepared for use by a server, and argument *qlength* specifies the length of the request queue for that socket. After the call, the system will enqueue up to *qlength* requests for connections. If the queue is full when a request arrives, the operating system will refuse the connection by discarding the request. *Listen* applies only to sockets that have selected reliable stream delivery service.

22.14 How A Server Accepts Connections

As we have seen, a server process uses the functions *socket*, *bind*, and *listen* to create a socket, bind it to a well-known protocol port, and specify a queue length for connection requests. Note that the call to *bind* associates the socket with a well-known protocol port, but that the socket is not connected to a specific foreign destination. In fact, the foreign destination must specify a *wildcard*, allowing the socket to receive connection requests from an arbitrary client.

Once a socket has been established, the server needs to wait for a connection. To do so, it uses function *accept*. A call to *accept* blocks until a connection request arrives. It has the form:

`newsock = accept(socket, addr, addrlen)`

Argument *socket* specifies the descriptor of the socket on which to wait. Argument *addr* is a pointer to a structure of type *sockaddr*, and *addrlen* is a pointer to an integer. When a request arrives, the system fills in argument *addr* with the address of the client that has placed the request and sets *addrlen* to the length of the address. Finally, the system creates a new socket that has its destination connected to the requesting client, and returns the new socket descriptor to the caller. The original socket still has a wildcard foreign destination, and it still remains open. Thus, the master server can continue to accept additional requests at the original socket.

When a connection request arrives, the call to *accept* returns. The server can either handle requests iteratively or concurrently. In the iterative approach, the server handles the request itself, closes the new socket, and then calls *accept* to obtain the next connection request. In the concurrent approach, after the call to *accept* returns, the master server creates a slave to handle the request (in UNIX terminology, it forks a child process to handle the request). The slave process inherits a copy of the new socket, so it can proceed to service the request. When it finishes, the slave closes the socket and terminates. The original (master) server process closes its copy of the new socket after starting the slave. It then calls *accept* to obtain the next connection request.

The concurrent design for servers may seem confusing because multiple processes will be using the same local protocol port number. The key to understanding the mechanism lies in the way underlying protocols treat protocol ports. Recall that in TCP a pair of endpoints define a connection. Thus, it does not matter how many processes use a given local protocol port number as long as they connect to different destinations. In the case of a concurrent server, there is one process per client and one additional process that accepts connections. The socket the master server process uses has a wildcard for the foreign destination, allowing it to connect with an arbitrary foreign site. Each remaining process has a specific foreign destination. When a TCP segment arrives, it will be sent to the socket connected to the segment's source. If no such socket exists, the segment will be sent to the socket that has a wildcard for its foreign destination. Furthermore, because the socket with a wildcard foreign destination does not have an open connection, it will only honor TCP segments that request a new connection.

22.15 Servers That Handle Multiple Services

The socket API provides another interesting possibility for server design because it allows a single process to wait for connections on multiple sockets. The system call that makes the design possible is called *select*, and it applies to I/O in general, not just to communication over sockets†. *Select* has the form:

$$\text{nready} = \text{select}(\text{ndesc}, \text{indesc}, \text{outdesc}, \text{excdesc}, \text{timeout})$$

In general, a call to *select* blocks waiting for one of a set of file descriptors to become ready. Argument *ndesc* specifies how many descriptors should be examined (the descriptors checked are always 2 through *ndesc*-1). Argument *indesc* is a pointer to a

†The version of *select* in Windows Sockets applies only to socket descriptors.

bit mask that specifies the file descriptors to check for input, argument *outdesc* is a pointer to a bit mask that specifies the file descriptors to check for output, and argument *excdesc* is a pointer to a bit mask that specifies the file descriptors to check for exception conditions. Finally, if argument *timeout* is nonzero, it is the address of an integer that specifies how long to wait for a connection before returning to the caller. A zero value for *timeout* forces the call to block until a descriptor becomes ready. Because the *timeout* argument contains the address of the timeout integer and not the integer itself, a process can request zero delay by passing the address of an integer that contains zero (i.e., a process can poll to see if I/O is ready).

A call to *select* returns the number of descriptors from the specified set that are ready for I/O. It also changes the bit masks specified by *indesc*, *outdesc*, and *excdesc* to inform the application which of the selected file descriptors are ready. Thus, before calling *select*, the caller must turn on those bits that correspond to descriptors to be checked. Following the call, all bits that remain set to 1 correspond to a ready file descriptor.

To communicate over more than one socket at a time, a process first creates all the sockets it needs and then uses *select* to determine which of them becomes ready for I/O first. Once it finds a socket has become ready, the process uses the input or output procedures defined above to communicate.

22.16 Obtaining And Setting Host Names

Most operating systems maintain an internal host name. For machines on the Internet, the internal name is usually chosen to be the domain name for the machine's main network interface. The *gethostname* function allows user processes to access the host name, and the *sethostname* function allows privileged processes to set the host name. *Gethostname* has the form:

`gethostname(name, length)`

Argument *name* gives the address of an array of bytes where the name is to be stored, and argument *length* is an integer that specifies the length of the *name* array. To set the host name, a privileged process makes a call of the form:

`sethostname(name, length)`

Argument *name* gives the address of an array where the name is stored, and argument *length* is an integer that gives the length of the name array.

22.17 Obtaining And Setting The Internal Host Domain

The operating system maintains a string that specifies the name domain to which a machine belongs. When a site obtains authority for part of the domain name space, it invents a string that identifies its piece of the space and uses that string as the name of the domain. For example, machines in the domain

`cs.purdue.edu`

have names taken from the Arthurian legend. Thus, one finds machines named *merlin*, *arthur*, *guenevere*, and *lancelot*. The domain itself has been named *camelot*, so the operating system on each host in the group must be informed that it resides in the *camelot* domain. To do so, a privileged process uses function *setdomainname*, which has the form:

`setdomainname(name, length)`

Argument *name* gives the address of an array of bytes that contains the name of a domain, and argument *length* is an integer that gives the length of the name.

User processes call *getdomainname* to retrieve the name of the domain from the system. It has the form:

`getdomainname(name, length)`

where argument *name* specifies the address of an array where the name should be stored, and argument *length* is an integer that specifies the length of the array.

22.18 Socket Library Calls

In addition to the functions described above, the socket API offers a set of library routines that perform useful functions related to networking. Figure 22.5 illustrates the difference between system calls and library routines. System calls pass control to the computer's operating system, while library routines are like other procedures that the programmer binds into a program.

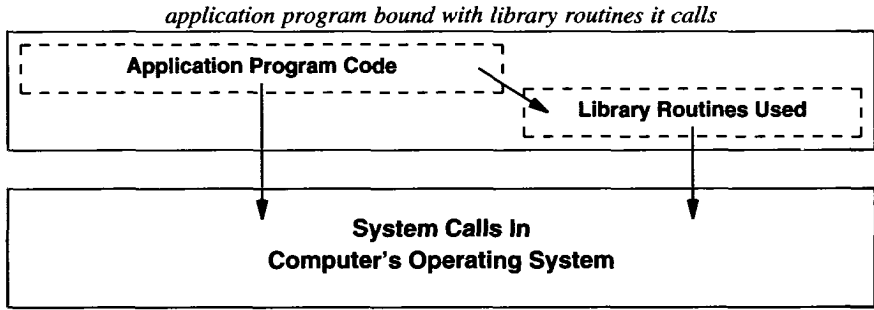


Figure 22.5 The difference between library routines, which are bound into an application program, and system calls, which are part of the operating system. A program can call either; library routines can call other library routines or system calls.

Many of the socket library routines provide database services that allow a process to determine the names of machines and network services, protocol port numbers, and other related information. For example, one set of library routines provides access to the database of network services. We think of entries in the services database as 3-tuples, where each 3-tuple contains the (human readable) name of a network service, the protocol that supports the service, and a protocol port number for the service. Library routines exist that allow a process to obtain information from an entry given any piece.

The next sections examine groups of library routines, explaining their purposes and providing information about how they can be used. As we will see, the sets of library routines that provide access to a sequential database follow a pattern. Each set allows the application to: establish a connection to the database, obtain entries one at a time, and close the connection. The routines used for these three operations are named *setXent*, *getXent*, and *endXent*, where *X* is the name of the database. For example, the library routines for the host database are named *sethostent*, *gethostent*, and *endhostent*. The sections that describe these routines summarize the calls without repeating the details of their use.

22.19 Network Byte Order Conversion Routines

Recall that machines differ in the way they store integer quantities and that the TCP/IP protocols define a machine independent standard for byte order. The socket API provides four library functions that convert between the local machine byte order and the network standard byte order. To make programs portable, they must be written to call the conversion routines every time they copy an integer value from the local machine to a network packet, or when they copy a value from a network packet to the local machine.

All four conversion routines are functions that take a value as an argument and return a new value with the bytes rearranged. For example, to convert a short (2-byte) integer from network byte order to the local host byte order, a programmer calls *ntohs* (network to host short). The format is:

$$\text{localshort} = \text{ntohs}(\text{netshort})$$

Argument *netshort* is a 2-byte (16-bit) integer in network standard byte order and the result, *localshort*, is in local host byte order.

The C programming language calls 4 byte (32 bit) integers *longs*. Function *ntohl* (network to host long) converts 4-byte longs from network standard byte order to local host byte order. Programs invoke *ntohl* as a function, supplying a long integer in network byte order as an argument:

$$\text{locallong} = \text{ntohl}(\text{netlong})$$

Two analogous functions allow the programmer to convert from local host byte order to network byte order. Function *htons* converts a 2-byte (short) integer in the host's local byte order to a 2-byte integer in network standard byte order. Programs invoke *htons* as a function:

$$\text{netshort} = \text{htons}(\text{localshort})$$

The final conversion routine, *htonl*, converts long integers to network standard byte order. Like the others, *htonl* is a function:

$$\text{netlong} = \text{htonl}(\text{locallong})$$

It should be obvious that the conversion routines preserve the following mathematical relationships:

$$\text{netshort} = \text{htons}(\text{ntohs}(\text{netshort}))$$

and

$$\text{localshort} = \text{ntohs}(\text{htons}(\text{localshort}))$$

Similar relationships hold for the long integer conversion routines.

22.20 IP Address Manipulation Routines

Because many programs translate between 32-bit IP addresses and the corresponding dotted decimal notation, the socket library includes utility routines that perform the translation. Procedures *inet_addr* and *inet_network* both translate from dotted decimal

format to a 32-bit IP address in network byte order. *Inet_addr* forms a 32-bit host IP address; *inet_network* forms the network address with zeroes for the host part. They have the form:

```
address = inet_addr(string)
```

and

```
address = inet_network(string)
```

where argument *string* gives the address of an ASCII string that contains the number expressed in dotted decimal format. The dotted decimal form can have 1 to 4 segments of digits separated by periods (dots). If all 4 appear, each corresponds to a single byte of the resulting 32-bit integer. If less than 4 appear, the last segment is expanded to fill remaining bytes.

Procedure *inet_ntoa* performs the inverse of *inet_addr* by mapping a 32-bit integer to an ASCII string in dotted decimal format. It has the form:

```
str = inet_ntoa(internetaddr)
```

where argument *internetaddr* is a 32-bit IP address in network byte order, and *str* is the address of the resulting ASCII version.

Often programs that manipulate IP addresses must combine a network address with the local address of a host on that network. Procedure *inet_makeaddr* performs such a combination. It has the form:

```
internetaddr = inet_makeaddr(net, local)
```

Argument *net* is a 32-bit network IP address in host byte order, and argument *local* is the integer representing a local host address on that network, also in local host byte order.

Procedures *inet_netof* and *inet_lnaof* provide the inverse of *inet_makeaddr* by separating the network and local portions of an IP address. They have the form:

```
net = inet_netof(internetaddr)
```

and

```
local = inet_lnaof(internetaddr)
```

where argument *internetaddr* is a 32-bit IP address in network byte order, and the results are returned in host byte order.

22.21 Accessing The Domain Name System†

A set of five library procedures comprise the interface to the TCP/IP domain name system. Application programs that call these routines become clients of one domain name system, sending one or more servers requests and receiving responses.

The general idea is that a program makes a query, sends it to a server, and awaits an answer. Because many options exist, the routines have only a few basic parameters and use a global structure, *res*, to hold others. For example, one field in *res* enables debugging messages while another controls whether the code uses UDP or TCP for queries. Most fields in *res* begin with reasonable defaults, so the routines can be used without changing *res*.

A program calls *res_init* before using other procedures. The call takes no arguments:

```
res_init()
```

Res_init reads a file that contains information like the name of the machine that runs the domain name server and stores the results in global structure *res*.

Procedure *res_mkquery* forms a domain name query and places it in a buffer in memory. The form of the call is:

```
res_mkquery(op, dname, class, type, data, datalen, newrr, buffer, buflen)
```

The first seven arguments correspond directly to the fields of a domain name query. Argument *op* specifies the requested operation, *dname* gives the address of a character array that contains a domain name, *class* is an integer that gives the class of the query, *type* is an integer that gives the type of the query, *data* gives the address of an array of data to be included in the query, and *datalen* is an integer that gives the length of the data. In addition to the library procedures, the socket API provides application programs with definitions of symbolic constants for important values. Thus, programmers can use the domain name system without understanding the details of the protocol. The last two arguments, *buffer* and *buflen*, specify the address of an area into which the query should be placed and the integer length of the buffer area, respectively. Finally, in the current implementation, argument *newrr* is unused.

Once a program has formed a query, it calls *res_send* to send it to a name server and obtain a response. The form is:

```
res_send(buffer, buflen, answer, anslen)
```

Argument *buffer* is a pointer to memory that holds the message to be sent (presumably, the application called procedure *res_mkquery* to form the message). Argument *buflen* is an integer that specifies the length. Argument *answer* gives the address in memory into which a response should be written, and integer argument *anslen* specifies the length of the answer area.

†Chapter 24 considers the Domain Name System in detail.

In addition to routines that make and send queries, the socket library contains two routines that translate domain names between conventional ASCII and the compressed format used in queries. Procedure *dn_expand* expands a compressed domain name into a full ASCII version. It has the form:

`dn_expand(msg, eom, compressed, full, fullen)`

Argument *msg* gives the address of a domain name message that contains the name to be expanded, with *eom* specifying the end-of-message limit beyond which the expansion cannot go. Argument *compressed* is a pointer to the first byte of the compressed name. Argument *full* is a pointer to an array into which the expanded name should be written, and argument *fullen* is an integer that specifies the length of the array.

Generating a compressed name is more complex than expanding a compressed name because compression involves eliminating common suffixes. When compressing names, the client must keep a record of suffixes that have appeared previously. Procedure *dn_comp* compresses a full domain name by comparing suffixes to a list of previously used suffixes and eliminating the longest possible suffix. A call has the form:

`dn_comp(full, compressed, cmprlen, prevptrs, lastptr)`

Argument *full* gives the address of a full domain name. Argument *compressed* points to an array of bytes that will hold the compressed name, with argument *cmprlen* specifying the length of the array. The argument *prevptrs* is the address of an array of pointers to previously compressed suffixes, with *lastptr* pointing to the end of the array. Normally, *dn_comp* compresses the name and updates *prevptrs* if a new suffix has been used.

Procedure *dn_comp* can also be used to translate a domain name from ASCII to the internal form without compression (i.e., without removing suffixes). To do so, a process invokes *dn_comp* with the *prevptrs* argument set to *NULL* (i.e., zero).

22.22 Obtaining Information About Hosts

Library procedures exist that allow a process to retrieve information about a host given either its domain name or its IP address. When used on a machine that has access to a domain name server, the library procedures make the process a client of the domain name system by sending a request to a server and waiting for a response. When used on systems that do not have access to the domain name system (e.g., a host not on the Internet), the routines obtain the desired information from a database kept on secondary storage.

Function *gethostbyname* takes a domain name and returns a pointer to a structure of information for that host. A call takes the form:

`ptr = gethostbyname(namestr)`

Argument *namestr* is a pointer to a character string that contains a domain name for the host. The value returned, *ptr*, points to a structure that contains the following information: the official host name, a list of aliases that have been registered for the host, the host address type (i.e., whether the address is an IP address), the address length, and a list of one or more addresses for the host. More details can be found in the UNIX Programmer's Manual.

Function *gethostbyaddr* produces the same information as *gethostbyname*. The difference between the two is that *gethostbyaddr* accepts a host address as an argument:

```
ptr = gethostbyaddr(addr, len, type)
```

Argument *addr* is a pointer to a sequence of bytes that contain a host address. Argument *len* is an integer that gives the length of the address, and argument *type* is an integer that specifies the type of the address (e.g., that it is an IP address).

As mentioned earlier, procedures *sethostent*, *gethostent*, and *endhostent* provide sequential access to the host database.

22.23 Obtaining Information About Networks

Hosts either use the domain name system or keep a simple database of networks in their internet. The socket library routines include five routines that allow a process to access the network database. Procedure *getnetbyname* obtains and formats the contents of an entry from the database given the domain name of a network. A call has the form:

```
ptr = getnetbyname(name)
```

where argument *name* is a pointer to a string that contains the name of the network for which information is desired. The value returned is a pointer to a structure that contains fields for the official name of the network, a list of registered aliases, an integer address type, and a 32-bit network address (i.e., an IP address with the host portion set to zero).

A process calls library routine *getnetbyaddr* when it needs to search for information about a network given its address. The call has the form:

```
ptr = getnetbyaddr(netaddr, addrtype)
```

Argument *netaddr* is a 32-bit network address, and argument *addrtype* is an integer that specifies the type of *netaddr*. Procedures *setnetent*, *getnetent*, and *endnetent* provide sequential access to the network database.

22.24 Obtaining Information About Protocols

Five library routines provide access to the database of protocols available on a machine. Each protocol has an official name, registered aliases, and an official protocol number. Procedure *getprotobyname* allows a caller to obtain information about a protocol given its name:

```
ptr = getprotobyname(name)
```

Argument *name* is a pointer to an ASCII string that contains the name of the protocol for which information is desired. The function returns a pointer to a structure that has fields for the official protocol name, a list of aliases, and a unique integer value assigned to the protocol.

Procedure *getprotobynumber* allows a process to search for protocol information using the protocol number as a key:

```
ptr = getprotobynumber(number)
```

Finally, procedures *getprotoent*, *setprotoent*, and *endprotoent* provide sequential access to the protocol database.

22.25 Obtaining Information About Network Services

Recall from Chapters 12 and 13 that some UDP and TCP protocol port numbers are reserved for well-known services. For example, TCP port 43 is reserved for the *whois* service. *Whois* allows a client on one machine to contact a server on another and obtain information about a user that has an account on the server's machine. The entry for *whois* in the services database specifies the service name, *whois*, the protocol, *TCP*, and the protocol port number 43. Five library routines exist that obtain information about services and the protocol ports they use.

Procedure *getservbyname* maps a named service onto a port number:

```
ptr = getservbyname(name, proto)
```

Argument *name* specifies the address of a string that contains the name of the desired service, and integer argument *proto* specifies the protocol with which the service is to be used. Typically, protocols are limited to TCP and UDP. The value returned is a pointer to a structure that contains fields for the name of the service, a list of aliases, an identification of the protocol with which the service is used, and an integer protocol port number assigned for that service.

Procedure *getservbyport* allows the caller to obtain an entry from the services database given the port number assigned to it. A call has the form:

```
ptr = getservbyport(port, proto)
```

Argument *port* is the integer protocol port number assigned to the service, and argument *proto* specifies the protocol for which the service is desired. As with other databases, a process can access the services database sequentially using *setservent*, *getservent*, and *endservent*.

22.26 An Example Client

The following example C program illustrates how a program uses the socket API to access TCP/IP protocols. It is a simple implementation of a *whois* client and server. As defined in RFC 954, the *whois* service allows a client on one machine to obtain information about a user on a remote system. In this implementation, the client is an application program that a user invokes with two arguments: the name of a remote machine and the name of a user on that machine about whom information is desired. The client calls *gethostbyname* to map the remote machine name into an IP address and calls *getservbyname* to find the well-known port for the *whois* service. Once it has mapped the host and service names, the client creates a socket, specifying that the socket will use reliable stream delivery (i.e., TCP). The client then binds the socket to the *whois* protocol port on the specified destination machine.

```
/* whoisclient.c - main */

#include <stdio.h>
#include <sys/types.h>
#include <sys/socket.h>
#include <netinet/in.h>
#include <netdb.h>

/*-----
 * Program:      whoisclient
 *
 * Purpose:      UNIX application program that becomes a client for the
 *               Internet "whois" service.
 *
 * Use:          whois hostname username
 *
 * Author:       Barry Shein, Boston University
 *
 * Date:         Long ago in a universe far, far away
 *-----
 */

main(argc, argv)
int argc;                /* standard UNIX argument declarations */
```

```

char *argv[];
{
    int s;                /* socket descriptor */
    int len;              /* length of received data */
    struct sockaddr_in sa; /* Internet socket addr. structure */
    struct hostent *hp;    /* result of host name lookup */
    struct servent *sp;    /* result of service lookup */
    char buf[BUFSIZ+1];    /* buffer to read whois information */
    char *myname;          /* pointer to name of this program */
    char *host;            /* pointer to remote host name */
    char *user;            /* pointer to remote user name */

    myname = argv[0];
    /*
     * Check that there are two command line arguments
     */
    if(argc != 3) {
        fprintf(stderr, "Usage: %s host username\n", myname);
        exit(1);
    }
    host = argv[1];
    user = argv[2];
    /*
     * Look up the specified hostname
     */
    if((hp = gethostbyname(host)) == NULL) {
        fprintf(stderr, "%s: %s: no such host?\n", myname, host);
        exit(1);
    }
    /*
     * Put host's address and address type into socket structure
     */
    bcopy((char *)hp->h_addr, (char *)&sa.sin_addr, hp->h_length);
    sa.sin_family = hp->h_addrtype;
    /*
     * Look up the socket number for the WHOIS service
     */
    if((sp = getservbyname("whois", "tcp")) == NULL) {
        fprintf(stderr, "%s: No whois service on this host\n", myname);
        exit(1);
    }
    /*
     * Put the whois socket number into the socket structure.
     */

```



```

sa.sin_port = sp->s_port;
/*
 * Allocate an open socket
 */
if((s = socket(hp->h_addrtype, SOCK_STREAM, 0)) < 0) {
    perror("socket");
    exit(1);
}
/*
 * Connect to the remote server
 */
if(connect(s, &sa, sizeof sa) < 0) {
    perror("connect");
    exit(1);
}
/*
 * Send the request
 */
if(write(s, user, strlen(user)) != strlen(user)) {
    fprintf(stderr, "%s: write error\n", myname);
    exit(1);
}
/*
 * Read the reply and put to user's output
 */
while( (len = read(s, buf, BUFSIZ)) > 0)
    write(1, buf, len);
close(s);
exit(0);
}

```

22.27 An Example Server

The example server is only slightly more complex than the client. The server listens on the well-known “whois” port and returns the requested information in response to a request from any client. The information is taken from the UNIX password file on the server’s machine.

```
/* whoisserver.c - main */
```

```

#include <stdio.h>
#include <sys/types.h>
#include <sys/socket.h>

```

```

#include <netinet/in.h>
#include <netdb.h>
#include <pwd.h>

/*-----
 * Program:      whoisserver
 *
 * Purpose:      UNIX application program that acts as a server for
 *               the "whois" service on the local machine.  It listens
 *               on well-known WHOIS port (43) and answers queries from
 *               clients.  This program requires super-user privilege to
 *               run.
 *
 * Use:          whois hostname username
 *
 * Author:       Barry Shein, Boston University
 *
 * Date:        Long ago in a universe far, far away
 *-----
 */

#define BACKLOG      5      /* # of requests we're willing to queue */
#define MAXHOSTNAME  32     /* maximum host name length we tolerate */

main(argc, argv)
int argc;                /* standard UNIX argument declarations */
char *argv[];
{
    int s, t;             /* socket descriptors */
    int i;                /* general purpose integer */
    struct sockaddr_in sa, isa; /* Internet socket address structure */
    struct hostent *hp;    /* result of host name lookup */
    char *myname;          /* pointer to name of this program */
    struct servent *sp;    /* result of service lookup */
    char localhost[MAXHOSTNAME+1]; /* local host name as character string */

    myname = argv[0];
    /*
     * Look up the WHOIS service entry
     */
    if((sp = getservbyname("whois", "tcp")) == NULL) {
        fprintf(stderr, "%s: No whois service on this host\n", myname);
        exit(1);
    }
}

```

```

}
/*
 * Get our own host information
 */
gethostname(localhost, MAXHOSTNAME);
if((hp = gethostbyname(localhost)) == NULL) {
    fprintf(stderr, "%s: cannot get local host info?\n", myname);
    exit(1);
}
/*
 * Put the WHOIS socket number and our address info
 * into the socket structure
 */
sa.sin_port = sp->s_port;
bcopy((char *)hp->h_addr, (char *)&sa.sin_addr, hp->h_length);
sa.sin_family = hp->h_addrtype;
/*
 * Allocate an open socket for incoming connections
 */
if((s = socket(hp->h_addrtype, SOCK_STREAM, 0)) < 0) {
    perror("socket");
    exit(1);
}
/*
 * Bind the socket to the service port
 * so we hear incoming connections
 */
if(bind(s, &sa, sizeof sa) < 0) {
    perror("bind");
    exit(1);
}
/*
 * Set maximum connections we will fall behind
 */
listen(s, BACKLOG);
/*
 * Go into an infinite loop waiting for new connections
 */
while(1) {
    i = sizeof isa;
    /*
     * We hang in accept() while waiting for new customers
     */
    if((t = accept(s, &isa, &i)) < 0) {

```

```

        perror("accept");
        exit(1);
    }
    whois(t);          /* perform the actual WHOIS service */
    close(t);
}
}
/*
 * Get the WHOIS request from remote host and format a reply.
 */
whois(sock)
int sock;
{
    struct passwd *p;
    char buf[BUFSIZ+1];
    int i;

    /*
     * Get one line request
     */
    if( (i = read(sock, buf, BUFSIZ)) <= 0)
        return;
    buf[i] = '\0';      /* Null terminate */
    /*
     * Look up the requested user and format reply
     */
    if((p = getpwnam(buf)) == NULL)
        strcpy(buf, "User not found\n");
    else
        sprintf(buf, "%s: %s\n", p->pw_name, p->pw_gecos);
    /*
     * Return reply
     */
    write(sock, buf, strlen(buf));
    return;
}

```

22.28 Summary

Because TCP/IP protocol software resides inside an operating system, the exact interface between an application program and TCP/IP protocols depends on the details of the operating system; it is not specified by the TCP/IP protocol standard. We examined the socket API, which was originally designed for BSD UNIX, but has become, de fac-

to, a standard used by vendors such as Microsoft. We saw that sockets adopted the UNIX open-read-write-close paradigm. To use TCP, a program must create a socket, bind addresses to it, accept incoming connections, and then communicate using the *read* or *write* primitives. Finally, when finished using a socket, the program must close it. In addition to the socket abstraction and system calls that operate on sockets, BSD UNIX includes library routines that help programmers create and manipulate IP addresses, convert integers between the local machine format and network standard byte order, and search for information such as network addresses.

The socket interface has become popular and is widely supported by many vendors. Vendors who do not offer socket facilities in their operating systems often provide a socket library that makes it possible for programmers to write applications using socket calls even though the underlying operating system uses a different set of system calls.

FOR FURTHER STUDY

Detailed information on the socket functions can be found in the *UNIX Programmer's Manual*, where Section 2 contains a description of each UNIX system call and Section 3 contains a description of each library procedure. UNIX also supplies on-line copies of the manual pages via the *man* command. Leffler, McKusick, Karels, and Quarterman [1989] explores the UNIX system in more detail. Hall et. al. [1993] contains the original standard for Windows Sockets, and Hall et. al. [1996] describes version 2.

Operating system vendors often provide libraries of procedures that emulate sockets on their systems. Consult vendors' programming manuals for details. Gilligan [RFC 2133] considers socket extensions for IPv6.

Volume 3 of this text describes how client and server programs are structured and how they use the socket API. The BSD sockets version of Volume 3 contains example code for Unix; the Windows sockets version contains the same examples for Microsoft Windows. The TLI version of Volume 3 provides an introduction to the *Transport Layer Interface*, an alternative to sockets used in System V UNIX.

EXERCISES

- 22.1 Try running the sample *whois* client and server on your local system.
- 22.2 Build a simple server that accepts multiple concurrent connections (to test it, have the process that handles a connection print a short message, delay a random time, print another message, and exit).

- 22.3** When is the *listen* call important?
- 22.4** What procedures does your local system provide to access the domain name system?
- 22.5** Devise a server that uses a single UNIX process, but handles multiple concurrent TCP connections. Hint: think of *select* (*poll* in SYSTEM V).
- 22.6** Read about the AT&T System V Transport Library Interface (TLI) and compare it to the socket interface. What are the major conceptual differences?
- 22.7** Each operating system limits the number of sockets a given program can use at any time. How many sockets can a program create on your local system?
- 22.8** The socket/file descriptor mechanism and associated *read* and *write* operations can be considered a form of object-oriented design. Explain why.
- 22.9** Consider an alternative interface design that provides an interface for every layer of protocol software (e.g., the system allows an application program to send and receive raw packets without using IP, or to send and receive IP datagrams without using UDP or TCP). What are the advantages of having such an interface? The disadvantages?
- 22.10** A client and server can both run on the same computer and use a TCP socket to communicate. Explain how it is possible to build a client and server that can communicate on a single machine without learning the host's IP address.
- 22.11** Experiment with the sample server in this chapter to see if you can generate TCP connections sufficiently fast to exceed the backlog the server specifies. Do you expect incoming connection requests to exceed the backlog faster if the server operates on a computer that has 1 processor than on a computer that has 5 processors? Explain.

23

Bootstrap And Autoconfiguration (BOOTP, DHCP)

23.1 Introduction

This chapter shows how the client-server paradigm is used for bootstrapping. Each computer attached to a TCP/IP internet needs to know its IP address before it can send or receive datagrams. In addition, a computer needs other information such as the address of a router, the subnet mask to use, and the address of a name server. Chapter 6 describes how a computer can use the RARP protocol at system startup to determine its IP address. This chapter discusses an alternative: two closely-related bootstrap protocols that each allows a host to determine its IP address without using RARP. Surprisingly, the client and server communicate using UDP, the User Datagram Protocol described in Chapter 12.

What makes the bootstrapping procedure surprising is that UDP relies on IP to transfer messages, and it might seem impossible that a computer could use UDP to find an IP address to use when communicating. Examining the protocols will help us understand how a computer can use the special IP addresses mentioned in Chapter 4 and the flexibility of the UDP/IP transport mechanism. We will also see how a server assigns an IP address to a computer automatically. Such assignment is especially important in environments that permit temporary internet connections or where computers move from one network to another (e.g., an employee with a portable computer moves from one location in a company to another).

23.2 The Need For An Alternative To RARP

Chapter 6 presents the problem diskless computers face during system startup. Such machines usually contain a startup program in nonvolatile storage (e.g., in ROM). To minimize cost and keep parts interchangeable, a vendor uses exactly the same program in all machines. Because computers with different IP addresses run the same boot program, the code cannot contain an IP address. Thus, a diskless machine must obtain its IP address from another source. In fact, a diskless computer needs to know much more than its IP address. Usually, the ROM only contains a small startup program, so the diskless computer must also obtain an initial memory image to execute. In addition, each diskless machine must determine the address of a file server on which it can store and retrieve data, and the address of the nearest IP router.

The RARP protocol of Chapter 6 has three drawbacks. First, because RARP operates at a low level, using it requires direct access to the network hardware. Thus, it may be difficult or impossible for an application programmer to build a server. Second, although RARP requires a packet exchange between a client machine and a computer that answers its request, the reply contains only one small piece of information: the client's 4-octet IP address. This drawback is especially annoying on networks like an Ethernet that enforce a minimum packet size because additional information could be sent in the response at no additional cost. Third, because RARP uses a computer's hardware address to identify the machine, it cannot be used on networks that dynamically assign hardware addresses.

To overcome some of the drawbacks of RARP, researchers developed the *BOOTstrap Protocol (BOOTP)*. Later, the *Dynamic Host Configuration Protocol (DHCP)* was developed as a successor to BOOTP. Because the two protocols are closely related, most of the description in this chapter applies to both. To simplify the text, we will describe BOOTP first, and then see how DHCP extends the functionality to provide dynamic address assignment.

Because it uses UDP and IP, BOOTP can be implemented with an application program. Like RARP, BOOTP operates in the client-server paradigm and requires only a single packet exchange. However, BOOTP is more efficient than RARP because a single BOOTP message specifies many items needed at startup, including a computer's IP address, the address of a router, and the address of a server. BOOTP also includes a vendor-specific field in the reply that allows hardware vendors to send additional information used only for their computers†.

23.3 Using IP To Determine An IP Address

We said that BOOTP uses UDP to carry messages and that UDP messages are encapsulated in IP datagrams for delivery. To understand how a computer can send BOOTP in an IP datagram before the computer learns its IP address, recall from Chapter 4 that there are several special-case IP addresses. In particular, when used as a destination address, the IP address consisting of all 1s (255.255.255.255) specifies limit-

†As we will see, the term "vendor-specific" is a misnomer because the current specification also recommends using the vendor-specific area for general purpose information such as subnet masks; DHCP changes the name of the field to *options*.

ed broadcast. IP software can accept and broadcast datagrams that specify the limited broadcast address even before the software has discovered its local IP address information. The point is that:

An application program can use the limited broadcast IP address to force IP to broadcast a datagram on the local network before IP has discovered the IP address of the local network or the machine's IP address.

Suppose client machine *A* wants to use BOOTP to find bootstrap information (including its IP address) and suppose *B* is the server on the same physical net that will answer the request. Because *A* does not know *B*'s IP address or the IP address of the network, it must broadcast its initial BOOTP request using the IP limited broadcast address. What about the reply? Can *B* send a directed reply? No, not usually. Although it may not be obvious, *B* may need to use the limited broadcast address for its reply, even though it knows *A*'s IP address. To see why, consider what happens if an application program on *B* attempts to send a datagram using *A*'s IP address. After routing the datagram, IP software on *B* will pass the datagram to the network interface software. The interface software must map the next hop IP address to a corresponding hardware address, presumably using ARP as described in Chapter 5. However, because *A* has not yet received the BOOTP reply, it does not recognize its IP address, so it cannot answer *B*'s ARP request. Therefore, *B* has only two alternatives: either broadcast the reply or use information from the request packet to manually add an entry to its ARP cache. On systems that do not allow application programs to modify the ARP cache, broadcasting is the only solution.

23.4 The BOOTP Retransmission Policy

BOOTP places all responsibility for reliable communication on the client. We know that because UDP uses IP for delivery, messages can be delayed, lost, delivered out of order, or duplicated. Furthermore, because IP does not provide a checksum for data, the UDP datagram could arrive with some bits corrupted. To guard against corruption, BOOTP requires that UDP use checksums. It also specifies that requests and replies should be sent with the *do not fragment* bit set to accommodate clients that have too little memory to reassemble datagrams. BOOTP is also constructed to allow multiple replies; it accepts and processes the first.

To handle datagram loss, BOOTP uses the conventional technique of *timeout and retransmission*. When the client transmits a request, it starts a timer. If no reply arrives before the timer expires, the client must retransmit the request. Of course, after a power failure all machines on a network will reboot simultaneously, possibly overrunning the BOOTP server(s) with requests. If all clients use exactly the same retransmission timeout, many or all of them will attempt to retransmit simultaneously. To avoid the resulting collisions, the BOOTP specification recommends using a random delay. In

addition, the specification recommends starting with a random timeout value between 0 and 4 seconds, and doubling the timer after each retransmission. After the timer reaches a large value, 60 seconds, the client does not increase the timer, but continues to use randomization. Doubling the timeout after each retransmission keeps BOOTP from adding excessive traffic to a congested network; the randomization helps avoid simultaneous transmissions.

23.5 The BOOTP Message Format

To keep an implementation as simple as possible, BOOTP messages have fixed-length fields, and replies have the same format as requests. Although we said that clients and servers are programs, the BOOTP protocol uses the terms loosely, referring to the machine that sends a BOOTP request as the *client* and any machine that sends a reply as a *server*. Figure 23.1 shows the BOOTP message format.

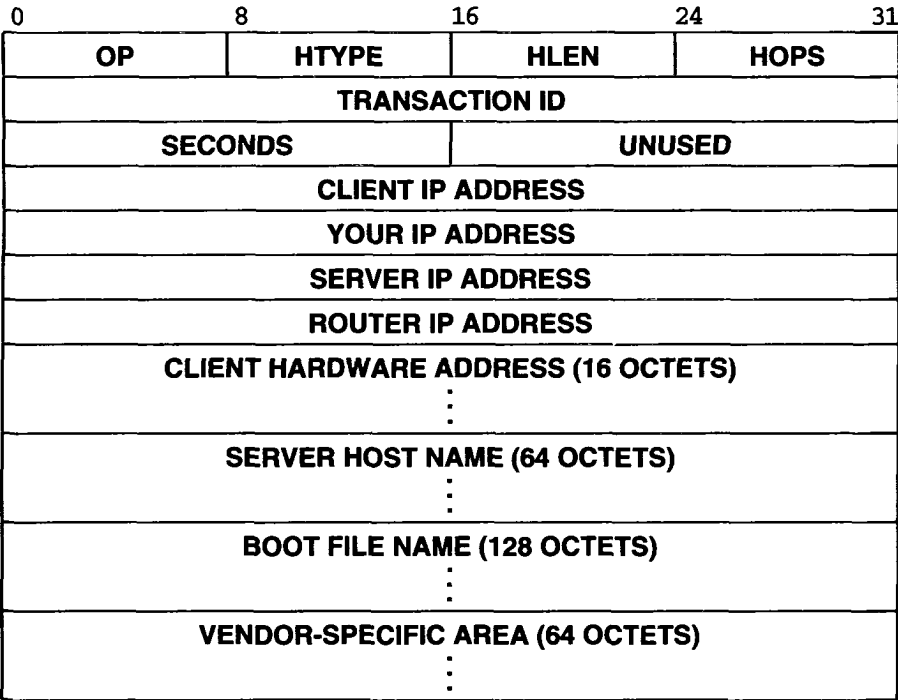


Figure 23.1 The format of a BOOTP message. To keep implementations small enough to fit in ROM, all fields have fixed length.

Field *OP* specifies whether the message is a request (1) or a reply (2). As in ARP, fields *HTYPE* and *HLEN* specify the network hardware type and length of the hardware address (e.g., Ethernet has type 1 and address length 6)[†]. The client places 0 in the *HOPS* field. If it receives the request and decides to pass the request on to another machine (e.g., to allow bootstrapping across multiple routers), the BOOTP server increments the *HOPS* count. The *TRANSACTION ID* field contains an integer that diskless machines use to match responses with requests. The *SECONDS* field reports the number of seconds since the client started to boot.

The *CLIENT IP ADDRESS* field and all fields following it contain the most important information. To allow the greatest flexibility, clients fill in as much information as they know and leave remaining fields set to zero. For example, if a client knows the name or address of a specific server from which it wants information, it can fill in the *SERVER IP ADDRESS* or *SERVER HOST NAME* fields. If these fields are nonzero, only the server with matching name/address will answer the request; if they are zero, any server that receives the request will reply.

BOOTP can be used from a client that already knows its IP address (e.g., to obtain boot file information). A client that knows its IP address places it in the *CLIENT IP ADDRESS* field; other clients use zero. If the client's IP address is zero in the request, a server returns the client's IP address in the *YOUR IP ADDRESS* field.

23.6 The Two-Step Bootstrap Procedure

BOOTP uses a two-step bootstrap procedure. It does not provide clients with a memory image — it only provides the client with information needed to obtain an image. The client then uses a second protocol (e.g., TFTP from Chapter 26) to obtain the memory image. While the two-step procedure many seem unnecessary, it allows a clean separation of configuration and storage. A BOOTP server does not need to run on the same machine that stores memory images. In fact, the BOOTP server operates from a simple database that only knows the names of memory images.

Keeping configuration separate from storage is important because it allows administrators to configure sets of machines so they act identically or independently. The *BOOT FILE NAME* field of a BOOTP message illustrates the concept. Suppose an administrator has several workstations with different hardware architectures, and suppose that when users boot one of the workstations, they either choose to run UNIX or a local operating system. Because the set of workstations includes multiple hardware architectures, no single memory image will operate on all machines. To accommodate such diversity, BOOTP allows the *BOOT FILE NAME* field in a request to contain a generic name like “unix,” which means, “I want to boot the UNIX operating system for this machine.” The BOOTP server consults its configuration database to map the generic name into a specific file name that contains the UNIX memory image appropriate for the client hardware, and returns the specific (i.e., fully qualified) name in its reply. Of course, the configuration database also allows completely automatic bootstrapping in which the client places zeros in the *BOOT FILE NAME* field, and BOOTP selects a

[†]Values for the *HTYPE* field can be found in the latest Assigned Numbers RFC.

memory image for the machine. The advantage of the automatic approach is that it allows users to specify generic names that work on any machine; they do not need to remember specific file names or hardware architectures.

23.7 Vendor-Specific Field

The *VENDOR-SPECIFIC AREA* contains optional information to be passed from the server to the client. Although the syntax is intricate, it is not difficult. The first four octets of the field are called a *magic cookie* and define the format of remaining items; the standard format described here uses a magic cookie value of 99.130.83.99 (dotted decimal notation). A list of items follows the cookie, where each item contains a one-octet *type*, an optional one-octet *length*, and a multi-octet *value*[†]. The standard defines the following types that have predetermined, fixed length values:

Item Type	Item Code	Value Length	Contents of Value
Padding	0	-	Zero - used only for padding
Subnet Mask	1	4	Subnet mask for local net
Time of Day	2	4	Time of day in universal time
End	255	-	End of item list

Figure 23.2 Items in the vendor information. The length field must exist for types 1 and 2; it must not exist for types 0 and 255.

Although a computer can obtain subnet mask information with an ICMP request, the standard now recommends that BOOTP servers supply the subnet mask in each reply to eliminate unnecessary ICMP messages.

Additional items in the *VENDOR-SPECIFIC AREA* all use a TLV encoding — each item has a *type* octet, *length* octet, and a *value*. Figure 23.3 lists the possibilities.

23.8 The Need For Dynamic Configuration

BOOTP was designed for a relatively static environment in which each host has a permanent network connection. A manager creates a BOOTP configuration file that specifies a set of BOOTP parameters for each host. The file does not change frequently because the configuration usually remains stable. Typically, a configuration continues unchanged for weeks.

With the advent of wireless networking and portable computers such as laptops and notebooks, it has become possible to move a computer from one location to another quickly and easily. BOOTP does not adapt to such situations because configuration information cannot be changed quickly. BOOTP only provides a static mapping from a host identifier to parameters for the host. Furthermore, a manager must enter a set of

[†]The format is an example of *TLV* encoding, which stands for *Type Length Value*.